Optimus - Low-Level Design

Version 2.45 — Plan 12.4 Auto mode, 2026-10-27; local filing — not activated.

Predecessor: Optimus-Cost-Agent-LLD-v2.44.pdf; SHA-256 80305863042581aa1bde2517c3d9473923bcd20db2c939a2925163592c63f885. Retain the predecessor unchanged in `docs/archive/`.

Native carried page bodies originate from Optimus-Cost-Agent-LLD-v2.41.pdf (SHA-256 69400fd474eb30711fcc9a061243d6a4d2e35d39d7794d4aa69f5ff51b98109b). This complete successor retains the predecessor's carried bodies and source corrections except for the changes identified below. Page keys remain those of the native baseline.

What this edition changes. It specifies the Auto composition and approval binding, frozen classifier input, `AutoRouter`, H, the Gateway decisions route, classification settlement, per-launch model stops, per-turn notices and routing records (§13.10). It also corrects the registry and production-construction statements those contracts depend on. New implementation statements describe main `bf440cba5c184f8bca7ed8af0a0ae6bb824f14f1`, PR #428's merge of main `8713dcbfbc05f9372619dfe655f83b247dc3751a` and PR head `df7dc3c52dbc64ec5d18d27e05766c1953715853` (2026-10-27). ADR-028 accepts ADR-007 with H; ADR-009 rung 2 remains deferred under #397.

Inspected production facts at bf440cb. Trusted registry enforcement is active. Ordinary bootstrap supplies the route policy and, with an eligible summarizer, the Context Engine attachment. Models carry their admission evidence as configuration; the trusted operator override participates in the approved effective hash. Auto adds its overlay and `_composition` binding. The classifier uses authenticated `POST /v1/decisions`, with independent admission and reply checks, and its attempts enter ordinary turn settlement as `classification`. An Auto turn's selected planning/answer model and context limits remain bound through planning and approved application; the summarizer retains its separate registry role.

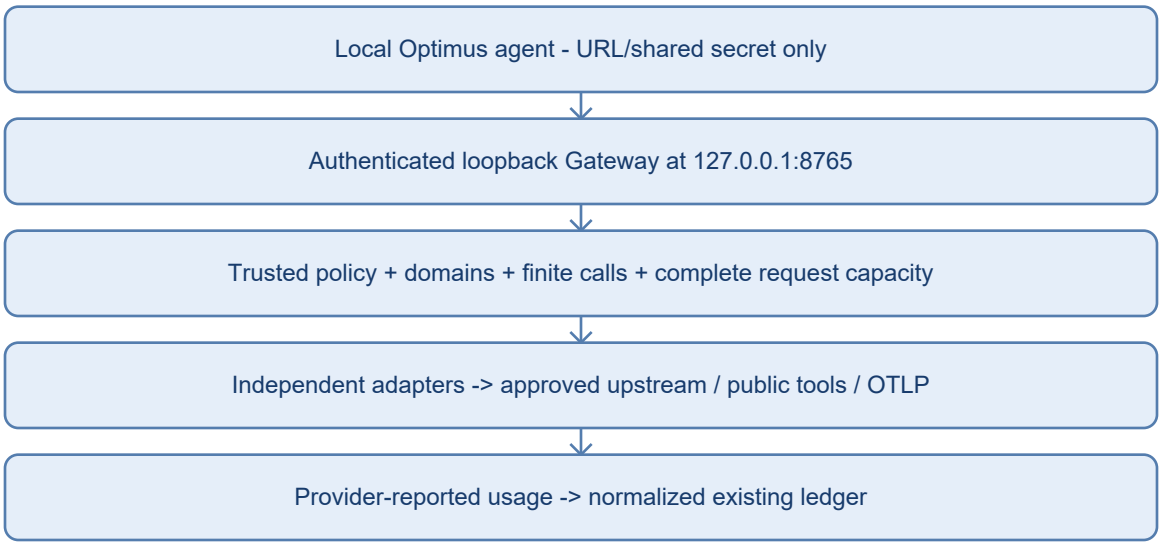
Provenance and controlling Plan 12.2 amendment. Unchanged baseline statements retain `91fd64ac78025efe1076bddb106ff89e6adb2030` provenance; §4B and the localized EDIT corrections retain `57e2069414fef32c6a6e4de42bd0dc7fb9f4f1ac`. This is not a full current-state refresh. The 2026-10-18 amendment, `docs/sources/plan-12-2-ordinary-shipping/architecture-amendment.md`, remains controlling for the unrefreshed Plan 12.2 material in §13 and §14, including compaction, profiles, maintenance and historical evidence. This edition supplies the newer registry, production-construction and Auto statements where expressly updated. Guardrails is unchanged. ADR-024 governs interactive loop control; retained numerical limits and legacy goal-loop descriptions do not override it.

Evidence stays attached to its source. The prior offline and EDIT receipts retain their original pins. Auto's three live acpx slices are reported at PR head `df7dc3c`, not as merge-commit reruns (§14). Pilot acceptance and source inspection do not establish every live failure path or a universal request-fit guarantee.

Authority. Codex's architecture wording; local filing by Claude under the operator's 2026-10-27 instruction: "Start the HLD v2.22 / LLD v2.45 successor for Plan 12.4 under #395, built from the baseline page keys with Codex authoring the replace-edits. SKIP=check-added-large-files is approved for its PDFs." Publication to main requires the operator's separate approval. The filing label does not disable Auto already shipped in the pinned code. This document grants no additional implementation, service-change, paid-call or publication authority. No product suite or live call was run to author this text.

0. Optimus AI Gateway architecture

The Gateway is a deterministic strict-loopback process. Agent-facing configuration remains only OPTIMUS_GATEWAY_URL and OPTIMUS_API_KEY. The Gateway holds the approved developer aggregator credential in its own local environment or OS credential storage. No hosted Optimus service, OAuth/device flow, tenant wallet, Vault or public origin is introduced. OpenRouter remains the default aggregator. Vercel is an optional second OpenAI-compatible endpoint by design; no Vercel integration exists at 91fd64a. Direct provider adapters remain removed.



Ingress / adapter	Downstream responsibility
/v1/responses; /v1/chat/completions	One approved OpenAI-compatible model transport; preserve distinct input/messages shapes
/v1/tools/web/search	Separate authorized search request. Tavily-backed at 91fd64a (Gateway-only `TAVILY_API_KEY`); the deterministic aggregator-plugin contract in §9C.1 is the target
/v1/tools/web/extract	Bounded HTTPS extract with provenance and revalidation
/v1/tools/package/lookup	Public PyPI/npm/Maven clients; independence from search configuration is required by §9C.4 and not yet implemented
/v1/tools/security/advisory	Public OSV client; same §9C.4 requirement
/v1/observability/traces	Structured trace validation/redaction -> OTel/OTLP -> Phoenix by default

All seven routes authenticate locally. No MCP endpoint is added. Product dollar stops retire only under the accepted migration; independent evaluation ceilings remain.

External component	Preserved edge
OpenRouter	Approved default model transport; separately authorized search is a target
Tavily	Implemented web-search provider at 91fd64a; Gateway-only key; replacement pending
Vercel	Design option; no integration at the inspected source
PyPI / npm / Maven	Independent package APIs
OSV	Independent advisory API
Phoenix / OTLP collector	Sanitized configured trace export
RedisTimeSeries	Existing usage/state attribution

The host owns Agent planning count/time/failure bounds; the Gateway owns enabled trusted request policy and capacity.

Target contract after the named trusted-policy / Task 11 / coverage migration as applicable; the dated inactive production baseline is stated on the cover.

C. Gateway responsibilities

- Authenticate the local agent with a shared secret on strict loopback.
- Resolve model aliases through an approved OpenAI-compatible aggregator.
- Broker web search (Tavily at 91fd64a; deterministic aggregator plugin is the target), bounded extract, package lookup, and OSV advisory; route independence from search configuration is a §9C.4 requirement not yet met.
- Enforce execution mode, tool policy, domain rules, call caps, provenance, complete request capacity and finite work.
- Persist run/session/request/Gateway/provider request attribution.
- Isolate upstream credentials from the agent process.
- Accept structured trace ingress, map it to OTel, and export OTLP.

D. Gateway-facing API shape

```
# Responses API shape - top-level "input"
POST /v1/responses

# Chat Completions shape - "messages" array
POST /v1/chat/completions

# Typed tools
POST /v1/tools/web/search
POST /v1/tools/web/extract
POST /v1/tools/package/lookup
POST /v1/tools/security/advisory

# Authenticated structured trace ingress
POST /v1/observability/traces
```

All routes authenticate with the local shared secret. Trace ingress is operational, not a model/tool billing route. No MCP endpoint is added.

E. Process-scoped configuration boundary

Agent process	Gateway process
`OPTIMUS_GATEWAY_URL`	`OPTIMUS_LOCAL_GATEWAY_PROVIDER=openrouter`
`OPTIMUS_API_KEY`	`OPTIMUS_LOCAL_GATEWAY_PROVIDER_API_KEY`
	`TAVILY_API_KEY` (Gateway-only; current search provider)
No upstream or OTel key	`OPTIMUS_LOCAL_GATEWAY_SHARED_SECRET`
Strict loopback URL	domains, Redis URL, optional OTLP endpoint
No override surface	no hosted origin, tenant profile, or production mode

0A. Local vs. Gateway Configuration and Provider-Cost Mapping

Phase 1 runtime rule

The agent runtime allows only `OPTIMUS_GATEWAY_URL` and `OPTIMUS_API_KEY`. The Gateway owns the developer's approved aggregator credential. Local agent-side Tavily, OpenAI, OpenRouter, GLM, LangSmith, or other provider keys are rejected.

Provider-cost mapping

`OPTIMUS_API_KEY` is a local shared secret, not a wallet key. The developer funds the aggregator account directly. The Gateway records the aggregator response into one local ledger.

Call class	Required accounting
Model completion	aggregator, actual provider/model when returned, tokens/cache, billing units, provider-reported `cost_usd`, request IDs
Web search	Tavily provider at 91fd64a: provider-native `tavily_credits` units recorded as `ProviderUsage` with request identity; the deterministic aggregator plugin (separate minimal model call, structured citations, search-use accounting) is the target
Package/advisory	operational request evidence; no fabricated provider cost
Trace export	delivery state and trace IDs; no allocated/amortized request charge

```
Agent shared secret
  -> loopback Gateway trust, capacity and accounting
  -> developer aggregator account
  -> provider-reported usage and USD cost
  -> GatewayUsage + normalized local ledger
```

The USD rename is complete at 91fd64a: ACP result payloads carry `ledger_run_total_cost_usd` and `ledger_run_total_billing_units` (`optimus/acp/dispatcher.py`), and no credit-named ledger field remains under `src/`. The rename changed no USD semantics and introduced no cross-run policy.

Gateway Tool and Observability Endpoints

The agent never calls OpenRouter, Vercel, package registries, OSV, Phoenix, or another OTLP backend directly. It calls the loopback Gateway's typed endpoints.

```
POST /v1/responses
POST /v1/chat/completions
POST /v1/tools/web/search
POST /v1/tools/web/extract
POST /v1/tools/package/lookup
POST /v1/tools/security/advisory
POST /v1/observability/traces
```

Search is an authorized, separate, minimal OpenRouter plugin request. Package and advisory routes remain independent of the paid-search configuration.

The canonical observability path is:

```
agent structured trace event
  -> authenticated /v1/observability/traces
  -> Gateway validation and redaction
  -> OpenTelemetry span/event mapping
  -> OTLP export
  -> Phoenix by default
```

The agent has no backend credential or direct backend egress. LangSmith and amortized observability-cost accounting are not part of this architecture. The route list has no MCP endpoint.

1. ACP Protocol Framing & JSON-RPC Contract Specification

To prevent streaming fragmentation errors, the ACP interface enforces a strict HTTP-style byte framing header layout. Malformed payloads trigger immediate JSON-RPC standard error blocks.

A. Header, Buffer Size Limits & Serialization Standards

Every frame must prepend an explicit Content-Length prefix sequence. To prevent Denial of Service (DoS) attacks, the transport plane enforces a strict Maximum Header Size of 8 KB and a Maximum Body Size of 10 MB. Responses echo a matching structural correlation integer via the id field.

```
[INBOUND SYSTEM STREAM TRANSACTION]
Content-Length: 114

{
  "jsonrpc": "2.0",
  "id": 42,
  "method": "agent/prompt",
  "params": {
    "prompt": "Refactor...",
    "targetFile": "main.py"
  }
}
```

```
PARSE_ERROR = {
    "code": -32700,
    "message": "Parse error: Invalid JSON payload received."
}
INVALID_REQUEST = {
    "code": -32600,
    "message": "Invalid Request: Schema mapping parameters missing."
}
INTERNAL_ERROR = {
    "code": -32603,
    "message": "Internal error: Underlying task execution failure."
}
ID_COLLISION_ERROR = {
    "code": -32001,
    "message": "Server error: Request ID collision detected."
}
```

2. Cross-Platform Stream Transport Layer

```
import asyncio
import sys
import json
import re
from typing import Optional, Tuple
from json import JSONDecodeError
from concurrent.futures import ThreadPoolExecutor

class CrossPlatformStreamReader:
    def __init__(self, max_header_bytes: int = 8192,
                 max_body_bytes: int = 10485760):
        self.executor = ThreadPoolExecutor(max_workers=1)
        self.max_header = max_header_bytes
        self.max_body = max_body_bytes

    def _sync_blocking_read(self) -> Tuple[Optional[bytes], bool]:
        header = b""
        while b"\r\n\r\n" not in header:
            if len(header) >= self.max_header:
                return None, True
            char = sys.stdin.buffer.read(1)
            if not char:
                return b"", False
            header += char
        match = re.search(b"(?i)Content-Length\s*:\s*(\d+)", header)
        if not match or int(match.group(1)) > self.max_body:
            return None, True
        content_length = int(match.group(1))
        return sys.stdin.buffer.read(content_length), False

    async def read_frame(self) -> Tuple[Optional[dict], Optional[dict]]:
        loop = asyncio.get_running_loop()
        raw_bytes, bad_frame = await loop.run_in_executor(
            self.executor, self._sync_blocking_read
        )
        if bad_frame:
            return None, PARSE_ERROR
        if not raw_bytes:
            return None, None
        try:
            return json.loads(raw_bytes.decode("utf-8")), None
        except JSONDecodeError:
            return None, PARSE_ERROR
```

3. Queued Task Lifecycle, Backpressure Controls & Runtime Pooling

```
import httpx
import uuid
import redis.asyncio as aioredis

class ManagedTaskManager:
    def __init__(self, max_concurrent_prompts: int = 3,
                 execution_timeout_secs: int = 120):
        self.semaphore = asyncio.Semaphore(max_concurrent_prompts)
        self.timeout = execution_timeout_secs
        self.all_tracked_tasks = {}
        self.http_pool = None
        self.redis_pool = None

    async def initialize_runtime_pools(self, redis_url: str):
        if self.all_tracked_tasks:
            raise RuntimeError(
                "Cannot reinitialize pools while tasks are running."
            )
        if self.http_pool or self.redis_pool:
            if self.http_pool:
                await self.http_pool.aclose()
            if self.redis_pool:
                await self.redis_pool.disconnect()
        self.http_pool = httpx.AsyncClient(
            limits=httpx.Limits(
                max_connections=10, max_keepalive_connections=5
            ),
            timeout=httpx.Timeout(30.0),
        )
        self.redis_pool = aioredis.ConnectionPool.from_url(
            redis_url, max_connections=20, decode_responses=True
        )
```

```
    async def enqueue_and_execute(self, request_id: Optional[int],
                                  pipeline_coro_factory) -> dict:
        active_id = request_id if request_id is not None \
            else int(uuid.uuid4().int & 0x7FFFFFFF)
        if not self.http_pool or not self.redis_pool:
            return {
                "status": "ERROR",
                "error": {
                    "code": -32003,
                    "message": "Server error: Connection pools uninitialized.",
                },
            }
        if active_id in self.all_tracked_tasks:
            return {
                "status": "ERROR",
                "error": {
                    "code": -32001,
                    "message": "Server error: Request ID collision detected.",
                },
            }

        async def _orchestrated_run():
            async with self.semaphore:
                coro = pipeline_coro_factory(self.http_pool, self.redis_pool)
                return await asyncio.wait_for(coro, timeout=self.timeout)

        loop = asyncio.get_running_loop()
        wrapper_task = loop.create_task(_orchestrated_run())
        self.all_tracked_tasks[active_id] = wrapper_task
        try:
            result = await wrapper_task
            return {"status": "SUCCESS", "data": result}
        except asyncio.CancelledError:
            return {
                "status": "ERROR",
                "error": {"code": -32603, "message": "Task cancelled during shutdown."},
            }
        except asyncio.TimeoutError:
```

```
                "error": {"code": -32603, "message": "Execution timeout."},
            }
        finally:
            self.all_tracked_tasks.pop(active_id, None)

    async def graceful_shutdown(self):
        for task in list(self.all_tracked_tasks.values()):
            task.cancel()
        if self.all_tracked_tasks:
            await asyncio.gather(
                *self.all_tracked_tasks.values(), return_exceptions=True
            )
        self.all_tracked_tasks.clear()
        if self.http_pool:
            await self.http_pool.aclose()
        if self.redis_pool:
            await self.redis_pool.disconnect()
```

4. Behavioral Governance: modes, strategies and scope

The following representative contracts preserve the existing scope and mode vocabulary while retiring the HAIKU/PRO model-tier enum. Executable implementations remain in src/; these type shapes are design contracts.

Contract	Values / fields
ExecutionMode	PLAN; CHAT; AGENT. This existing vocabulary retains the read-only Plan/Chat boundary.
CodeGenerationScope	INLINE_SNIPPET; PATCH_PROPOSAL; FILE_MUTATION; MULTI_FILE_CHANGESET
ExecutionStrategy	DIRECT; PLAN_AND_EXECUTE; REACT; REFLECTION
RigorLevel	LOW; MEDIUM; HIGH
Registry price tier	ultra-cheap; cheap; review. These remain the registry's `Tier` values; they are not Auto's task classes.
Auto task tier / registry role	easy; medium; complex, mapped to the corresponding implementer roles. Other registry roles are escalation, summarizer, reviewer and classifier; reviewer remains dormant. An escalation assignment does not activate struggle escalation.
ToolOperationKind	READ; WRITE; EXTERNAL_MUTATION
OptimusToolErrorCode	VALIDATION_ERROR; RESOURCE_ERROR; PROVIDER_ERROR; TIMEOUT_ERROR
OptimusToolError	code; message; optional context. Exception keeps those fields and its message.
AssumptionItem	claim; basis; confidence default medium; requires_verification default true
AssumptionLedger	assumptions list, empty by default; no new conversation claim-tracking subsystem
ToolResponse	is_success; payload empty dict by default; optional error_code/error_msg

ToolResponse.to_jsonrpc_error maps VALIDATION_ERROR to -32002, RESOURCE_ERROR to -32004, PROVIDER_ERROR to -32006, TIMEOUT_ERROR to -32005, and unmapped/no typed code to -32603. Return code, Tool Execution Failure message, and data containing typed error_code and context. These fields are retained across the successor.

Target contract after the named trusted-policy / Task 11 / coverage migration as applicable; the dated inactive production baseline is stated on the cover.

4. Scope classification and strategy selection (continued)

The following retained prototype illustrates scope classification; it is not a new runtime activation or additional policy decision. Ordered branches:

Priority	Condition	Scope
1	Shared module touched; multiple directory roots; structural create/delete across more than one file; or more than three files	MULTI_FILE_CHANGESET
2	Structural create/delete; or one file with more than 100 lines	FILE_MUTATION
3	More than 15 lines	PATCH_PROPOSAL
4	Otherwise	INLINE_SNIPPET

Execution strategy remains harness-controlled. Start with at most five tool calls and zero reflection passes; a validation failure or shared-module change selects bounded Reflection with at most 15 calls and three passes. High task risk or a multi-file changeset selects Plan-and-Execute with at most ten calls and one reflection. High ambiguity otherwise selects ReAct with at most 12 calls. Other work uses Direct execution. No new mandatory reflection loop is introduced. At `bf440cb`, trusted registry enforcement is active. Ordinary launches use a configured eligible fixed model; approved Auto launches classify and bind a planning/answer model per turn (§13.10). Both obey final complete-request capacity, with a planning/answer ceiling of 262144 including one output reserve and the admitted route's smaller bounds where applicable. Explicit empirical admission does not establish a universal token bound. Retire the old branch-specific 100000/150000/250000 token ceilings and HAIKU/PRO aliases. History strategy allocations and final packing remain separate. These retained scope/strategy prototypes do not implement H or determine its task tier. Automatic struggle escalation remains deferred under #397. Mutation enforcement continues in section 4: PLAN cannot perform WRITE or EXTERNAL_MUTATION and raises the typed validation error. The existing MutationGuard wraps that assertion before execution; the continuation on the next page retains its path-containment contracts.

4. Mutation assertion and path containment (retained prototype)

The full assertion is shown here so the raise statement is not orphaned after the preceding page replacement. ExecutionMode is the retained PLAN/CHAT/AGENT vocabulary. These illustrative contracts preserve the original containment checks; executable implementation remains authoritative.

```
def assert_mutation_allowed(mode: ExecutionMode,
                            operation: ToolOperationKind, tool_name: str):
    if mode != ExecutionMode.AGENT and operation in {
        ToolOperationKind.WRITE, ToolOperationKind.EXTERNAL_MUTATION
    }:
        raise OptimusToolError(
            OptimusToolErrorCode.VALIDATION_ERROR,
            f"Access security violation error: "
            f"Mutation blocked in Plan/Chat mode: {operation}",
        )
class MutationGuard:
    def __init__(self, mode: ExecutionMode):
        self.mode = mode
    def enforce_gate(self, op_kind: ToolOperationKind, tool_name: str):
        assert_mutation_allowed(self.mode, op_kind, tool_name)
class ASTOptimizationInput(BaseModel):
    project_path: DirectoryPath = Field(
        ..., description="Absolute path to the project root."
    )
    target_file: str = Field(..., description="Relative file path string.")
    adl_rules: list[str] = Field(default_factory=list)
    execution_mode: ExecutionMode = Field(default=ExecutionMode.PLAN)
    generation_scope: CodeGenerationScope = Field(
        default=CodeGenerationScope.INLINE_SNIPPET
    )
    rigor_budget: RigorLevel = Field(default=RigorLevel.LOW)
    assumption_ledger: AssumptionLedger = Field(
        default_factory=AssumptionLedger
    )
    @model_validator(mode="after")
    def validate_paths_and_safeguards(self) -> "ASTOptimizationInput":
        if Path(self.target_file).is_absolute():
            raise ValueError("Target file must be a relative path configuration.")
        try:
            resolved_base = Path(os.path.abspath(self.project_path)).resolve(
                strict=True
            )
            candidate_path = Path(os.path.abspath(
                os.path.join(str(resolved_base), self.target_file)
            ))
        except (ValueError, FileNotFoundError, RuntimeError):
            raise ValueError("Malformed or invalid project root directory specification.")
        try:
            if candidate_path.exists() or candidate_path.is_symlink():
                resolved_target = candidate_path.resolve(strict=False)
                if os.path.commonpath([
                    str(resolved_base), str(resolved_target)
                ]) != str(resolved_base):
                    raise ValueError(
                        "Breakout attempt detected via target file path alignment."
                    )
            else:
                parent_path = candidate_path.parent
                resolved_parent = parent_path.resolve(strict=True)
                if os.path.commonpath([
                    str(resolved_base), str(resolved_parent)
                ]) != str(resolved_base):
                    raise ValueError(
                        "Symlink ancestry breakout via target parent folder."
                    )
        except ValueError as ve:
            raise ValueError(
                f"Cross-drive partition boundary constraint or layout failure: {ve}"
            )
        except (FileNotFoundError, RuntimeError):
            raise ValueError(
                "Missing or invalid ancestral directory infrastructure mapping."
            )
        return self
```

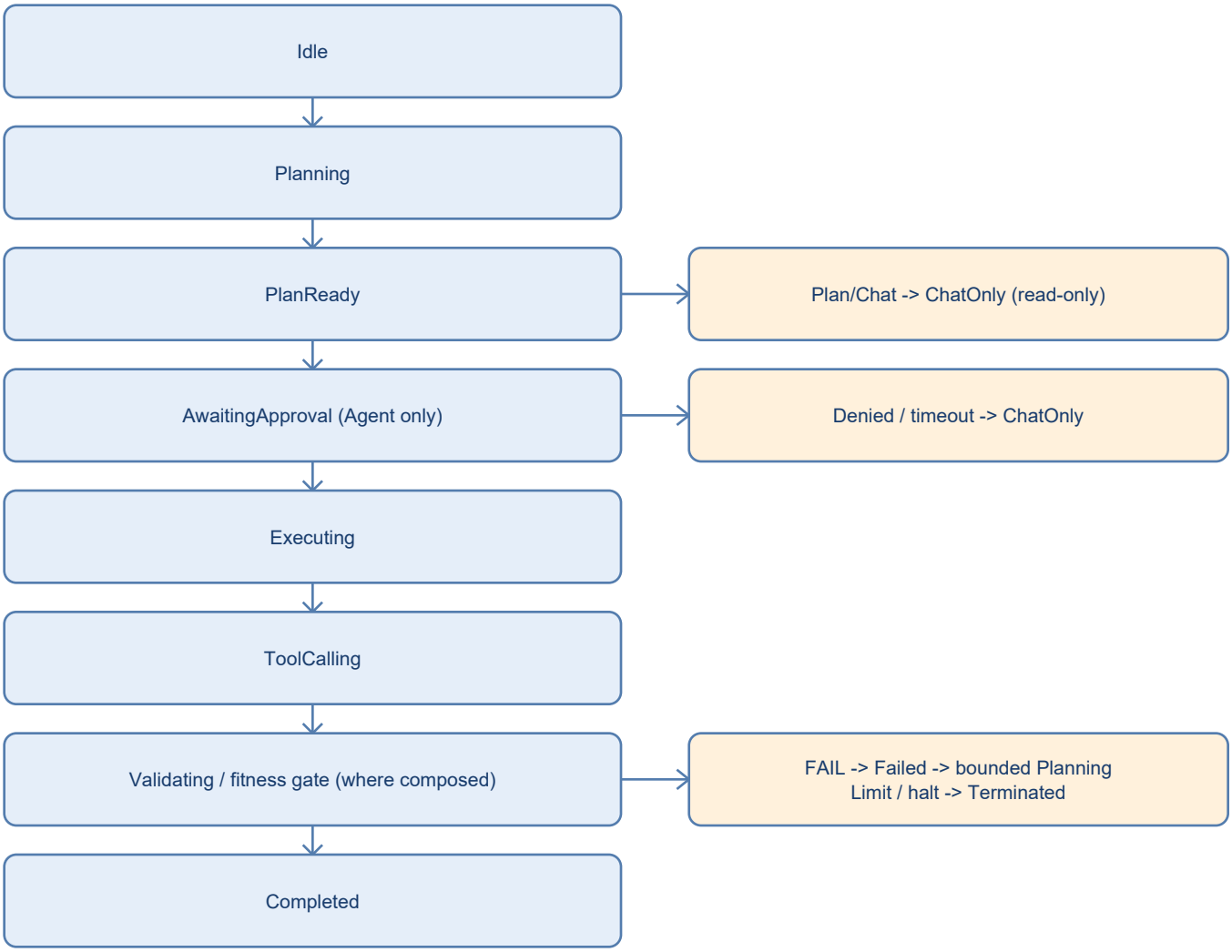
4A. Agent State Model and Execution Pathways

This section defines the complete agent lifecycle — from initial idle state through planning, execution, tool calling, and termination — and specifies which transitions are valid, which are rejected deterministically, and what permissions each mode grants.

Lifecycle State Machine

The agent operates as a deterministic state machine. Every state transition is validated at the control-plane boundary; invalid transitions are rejected with a structured -32002 error before any side-effectful work begins.

4A. Agent lifecycle state machine



For ACP WRITE/EDIT at `57e2069`, exact plan approval is followed by `assert_mutation_allowed` / `MutationGuard` and `PreToolGuard`, then a direct working-tree write. There is no Validating/fitness step or shadow execution before that write. Plan TESTs run afterwards in the working tree. An EDIT applicability failure stops the run without fitness-driven replanning (§4B).

Branch	Required transition
Plan/Chat	PlanReady -> ChatOnly; no mutation
Agent	PlanReady -> AwaitingApproval; current exact approval -> Executing
Tool result	ToolCalling -> Validating where fitness is composed; PASS -> Executing or Completed. The ACP WRITE/EDIT path is the exception below (§4B)
Fitness failure	Failed -> Planning only within finite count/time/failure bounds
Exhaustion / cancellation	Failed -> Terminated with true reason and retained effects/receipts

No direct PlanReady -> Executing transition is allowed. Historical approval and model prose cannot bypass the current artifact gate. This figure abstracts lifecycle and approval; Auto's implemented opt-in selection is specified in §13.10. Selecting Auto

does not bypass any mode or artifact-approval transition. Retained interim loop-limit labels are historical; ADR-024 governs current interactive loop control.

4A. Valid and invalid transitions

Every transition is deterministic; model text cannot override a rejection. Existing mode and exact-approval checks remain required.

From	To	Permitted / condition
Idle	Planning	Yes, user request
Planning	PlanReady	Yes, requirements and mode resolved
PlanReady	ChatOnly	Yes, Plan/Chat; no mutation
PlanReady	Executing directly	No; AwaitingApproval cannot be bypassed
PlanReady	AwaitingApproval	Yes, Agent needs exact artifact approval
AwaitingApproval	Executing	Yes, current approval and target checks pass
AwaitingApproval	ChatOnly	Yes, denied/timed-out approval returns advisory outcome
Executing	ToolCalling	Yes, ToolInvocationPolicy authorizes the call
Executing	Mutating ToolCalling	Only Agent mode plus approval/guard controls
ToolCalling	Validating	Yes, operation response received
Validating	Executing	Yes, applicable gates pass
Validating	Failed	Yes, record actual failure and count it
Failed	Planning	Only within configured count/time/failure controls; inject targeted failure context
Failed	Terminated	Finite work exhausted, cancellation or permanent failure; report cause. No product dollar exhaustion after migration.
Executing	Completed	All planned work/gates complete; actual effect evidence retained
Any	Executing by bypass	No, -32002; approval path remains mandatory

Interim ACP defaults remain three planning rounds, 30 minutes and two repeated failures. Generic architecture diagrams do not override concrete configured execution limits. Independent evaluation budget exhaustion is a separate scoped stop, not product policy.

Mode-specific permission matrix

Permissions are enforced by the harness, not model instruction. Agent permissions still require the current artifact, approval and tool-specific guard; a Yes cell is not unconditional authority.

Capability	Plan/Chat	Agent	Enforcement
Read repo / inspect files	Yes	Yes	Existing eligible workspace / read controls
Read-only analysis tools	Yes	Yes	Existing read controls
Policy-gated web search / extract	Yes	Yes	ToolInvocationPolicy and provenance
Produce diffs / snippets as text	Yes	Yes	Text payload only, not applied
Shadow apply patch	No	Yes	assert_mutation_allowed gate; not used by ACP WRITE/EDIT at `57e2069` (§4B)
Mutate working tree	No	Yes	ACP WRITE/EDIT: exact plan approval, then assert_mutation_allowed / MutationGuard and PreToolGuard, then direct working-tree write; no shadow execution or pre-write fitness check (§4B)
Shell / build / test	No	Yes	Agent mode, approval, applicable guards and fitness gates where composed; ACP plan TESTs run after the write in the working tree, not as pre-write fitness checks (§4B)
Append cost / audit telemetry	Yes	Yes	Append-only
Modify telemetry / ledger entries	No	No	Immutable after write

Execution pathway

The retained pathway connects captured intent/mode, eligible model selection, mutation gating and telemetry through the Gateway boundary. At `bf440cb`, Auto/classifier routing exists in the separately approved opt-in composition (§13.10); these lifecycle and permission figures do not specify that classifier. Auto changes planning/answer model selection, not mutation authority. ChatOnly, AwaitingApproval and the applicable failure/fitness branches retain their execution boundaries; ADR-024 governs interactive loop recovery.

Target contract after the named trusted-policy / Task 11 / coverage migration as applicable; the dated inactive production baseline is stated on the cover.

4A. Execution pathway

Stage	Deterministic boundary
IDE / ACP ingress	User prompt, mode/config capture and session identity
Host selection	Enforced trusted registry; configured fixed model ordinarily, or H and `AutoRouter` binding one eligible model per Auto turn (§13.10). The old hard-coded Haiku/Pro triage is not this mechanism
Mode and approval	Chat stays read-only; Agent needs exact artifact approval before effectful tools
Feedforward assembly	Bounded workspace evidence and Redis state plus the optional Context Engine; exact authority. AST slicing and ADL constraint controls are intended, not established at 91fd64a (§14)
Final Gateway guard	Strict loopback, shared secret, approved endpoint, full input and one enforced output reserve per actual attempt
Upstream generation	Gateway-owned aggregator credential; no Vault, hosted tenant or provider key in CORE
Feedback	Complete finish required for executable candidate. ACP WRITE/EDIT: exact approval, then assert_mutation_allowed / MutationGuard and PreToolGuard, then direct working-tree write and plan TESTs afterwards. No shadow execution or fitness check precedes the write; EDIT applicability failure stops the run without fitness-driven replanning (§4B)
Mutation gate	Current approved artifact/target, permission and start gates; no model authority
Telemetry / settlement	Actual provider receipts, exact effects, exactly-once costs and explicit unknowns; no fabricated charge

This table abstracts the planning/answer pathway. Auto's preceding classifier call and its separate accounting are specified in §13.10. Ordinary production construction exists at `bf440cb`; a test composition still proves only its own configuration. Existing mode and exact-approval boundaries remain. The Plan 12.2 amendment controls unrefreshed attachment details, and ADR-024 controls interactive loop behavior.

4A. Execution strategy selection and tool policy

Strategy	Trigger	Behavior / bounds
Direct	Low rigor, no mutation	Single result, up to five tool calls, zero reflection; eligible fixed model
Plan-and-Execute	High task risk or multi-file changeset (retained prototype)	Structured plan and exact approval; up to ten calls and one reflection
ReAct	High ambiguity	Interleave bounded reasoning/observations; preserve configured call/iteration bounds
Reflection	Gate failure or shared/core change	Targeted recovery, at most three reflection passes; never the default loop

These descriptive strategies do not select Auto. Auto is an independent opt-in launch composition (§13.10), and its task tiers are distinct from execution-strategy and rigor policy. The predecessor's three-round/30-minute/two-repeated-failure statement retains its historical pin; ADR-024 governs interactive loop control. Planning/answer work obeys final admitted route capacity, including explicit empirical-admission limitations; HAIKU/PRO aliases and separate risk-tier context ceilings remain retired. Tool invocation is harness-controlled, first matching policy wins. Reject mutating tools in Plan/Chat with -32002 before I/O. WEB_SEARCH requires a current/external-fact, API or security signal and a valid reason code; local evidence remains first. WEB_EXTRACT requires the URL to belong to the prior authorized search result set. A mutation without current artifact approval is rejected before execution. Local read/inspect/analysis tools still pass the existing deterministic mode/permission rules. The model never selects the policy row or invents provenance.

4A. Failure and retry lifecycle

Record the true failure, preserve receipts/effects and inject bounded failure context only when configured work limits permit another planning round. Keep count/time/repeated-failure controls distinct from provider attempts. The interim ACP defaults are three planning rounds, 30 minutes and two repeated failures. Product BudgetExhaustedError is retired by the accepted migration; independent evaluations may retain their scoped dollar stop. The legacy host permits an initial attempt plus at most three retries. The inactive-policy Gateway separately permits at most three upstream attempts, including its legacy transient timeout/URL/5xx retries. Once enforcement is active, the Gateway permits one primary attempt and at most one identical-payload resend on the same route, only after a definitely NOT_SENT result (including a proven pre-connection timeout) or rejected HTTP 429. A timeout after connection, uncertain delivery, 5xx or unknown accounting is not resent. The host honors non-retryable classifications so it cannot multiply attempts. No alternative-model recovery is added. Each physical attempt retains its own receipt.

4B. Agent-mode directive plan and localized EDIT

These contracts are implemented at main `57e2069` (#407 / PR #411, with the merged #410 / PR #413 and #412 / PR #414 changes). Source paths in this section are under `src/optimus/`.

4B.1 Final plan and grammar

Every Agent-mode plan comes from the multi-turn planning loop (`agent/runner.py:476–495`). The final-plan directive set is READ, WRITE, EDIT and TEST. WRITE replaces the whole file from an opaque body (literal EDIT headers and marker lines remain content), fitting LF/CRLF body breaks to the existing file's uniform style (LF if it has no terminators), retaining its UTF-8 BOM, and replacing the body's last terminator according to the existing file's final-newline convention while preserving earlier empty lines (an empty body clears the text but retains an existing BOM); a new file uses LF and ends as the body does, and a mixed-ending target, including one with a lone CR, is refused unchanged as `WRITE_LINE_ENDINGS_MIXED` (`agent/tools.py:108–142`; `tools/mutation_tools.py:95–111`; `tools/line_endings.py:18–44`). MCP directives are invalid in a final plan.

A model emits EDIT without a base digest:

```
EDIT <relative-path>
<<<<<<< OLD
<one or more OLD lines>
======= NEW
<zero or more NEW lines>
>>>>>>> END
```

Repeat the block for further changes to the same existing file. Each OLD contains at least one line; NEW may be empty to delete its OLD lines. Markers are whole lines compared after strip. They are reserved at directive positions and within EDIT parsing, with the WRITE-body exception above. A marker outside EDIT, out-of-order markers, an unterminated block, an empty OLD or zero blocks rejects the parse. Only CR, LF and CRLF end EDIT lines. Every non-EDIT plan uses the R1 `plan_lines` split: LF ends a line, CRLF stays paired, terminators are retained, and every other `str.splitlines()` separator is content. In that R1 split, `directive_text` prevents any line holding a content separator from becoming directive syntax. READ, WRITE, EDIT, TEST, MCP_LIST and MCP_CALL tokens use only spaces and tabs as separators and padding. If exact EDIT parsing did not return a valid EDIT, `_invents_an_edit` rejects an EDIT header visible only under `str.splitlines()` outside a WRITE body, even when that EDIT is malformed. One empty LF/CRLF line directly before the next recognized directive separates it from a WRITE body; other lines, including space-only lines, remain content. Planning-loop surface checks keep OLD/NEW bodies opaque and use R1 outside them (`agent/directives.py:10–25`, `:118–213`, `:265–305`, `:319–355`; `agent/planning_loop.py:633–649`).

The model must not supply `base_sha256` (`edit_base="forbidden"`). The harness rewrites only the header to `EDIT <relative-path> base_sha256=<64 lowercase hex>`. Re-parsing proves that nothing else changed (`agent/directives.py:244–276`; `agent/planning_loop.py:817–820`; `agent/runner.py:722–724`). Both prompt grammars document EDIT and the WRITE fitting/refusal rule under `AGENT_PLANNER_PROMPT_VERSION:2026-10-24-p407-edit-p410-write-line-endings` and `MULTI_TURN_PLANNER_PROMPT_VERSION:2026-10-24-p407-edit-p410-write-line-endings` (`agent/prompts.py:10–15`, `:40–50`).

4B.2 Evidence and binding before approval

The evidence digest is SHA-256 of the whole file's bytes when evidence was shown to the model. Turn-1 workspace context records it for each included file, whether shown whole or truncated (`agent/workspace_context.py:463–473`). A ranged READ records the whole-file `source_sha256` and `line_endings` from the same byte snapshot; its displayed READ_BLOCK header includes `eol=lf|crlf|mixed|none` (`loops/tools.py:112–130`; `agent/planning_loop.py:534–541`).

Only the final plan's turn counts. Initial-context digests are available only when that is planning turn 1, plus that turn's current reads (`agent/planning_loop.py:1951–1956`). The digest does not establish mechanical grounding of OLD; that remains #258.

`_finish_agent_planning` binds EDIT before calculating the plan hash. No evidence digest gives `EDIT_SOURCE_UNSEEN`; a changed file gives `EDIT_SOURCE_CHANGED`; a missing target with evidence gives `EDIT_TARGET_MISSING`. Binding failure occurs before the hash and approval. Approval text, plan hash and stored record use the bound text (`agent/runner.py:738–756`, `:1982–1994`). Approval shows the exact plan text, without an ACP diff.

4B.3 Apply and write

Fresh execution and stored-plan replay use the same sequence; replay never binds an EDIT (`agent/runner.py:1470–1536`):

1. Refuse an unbound EDIT with `EDIT_SOURCE_UNSEEN`.
2. Refuse a target that is not a file with `EDIT_TARGET_MISSING`.
3. Perform a gated raw read with `newline=""`. A decode error gives `EDIT_NOT_UTF8`.
4. Compare the base digest. A mismatch gives `EDIT_SOURCE_CHANGED`.
5. Run `apply_edit_blocks` against the base.
6. Perform the exact gated write through `AgentToolbox.write_file`, which forwards EDIT's `newline=""` and `expected_sha256` to `write_file` rather than the WRITE fitting adapter. The shared `_authorize_file_mutation` runs `assert_mutation_allowed(WRITE_FILE)` / `MutationGuard`, then `PreToolGuard` with `FILE_WRITE` and `generation_scope=FILE_MUTATION`. After the guard allows it, re-hash the target bytes immediately before writing. A mismatch raises `WriteSourceChanged`, a `MutationForbidden`, and gives `EDIT_SOURCE_CHANGED`. The write uses `write_text(encoding="utf-8", newline="")` (`agent/tools.py:104–147`; `tools/mutation_tools.py:67–92`, `:114–140`; `runtime/mutation.py:40–41`). The exact `write_file` primitive now defaults to `newline=""` for every caller.

This is a direct working-tree write after exact approval and guards. The ACP WRITE/EDIT path uses no diff parser, `ShadowWorkspaceMutationRunner`, shadow apply, rollback or pre-write fitness check. The shadow runner is constructed only in `retry/gated_run.py:103`; no production path constructs `GatedRetryRunner`. Plan TESTs run afterwards in the working tree (`agent/runner.py:869–907`, `:1190–1225`). The sequence is not atomic against concurrent external writers.

4B.4 Matching and byte preservation

`apply_edit_blocks` matches OLD whole lines exactly once, excluding terminators. Blocks must not overlap and are applied to the base, not successively to earlier replacement results. Only CR, LF and CRLF end a line (`agent/edits.py:55–110`).

A leading BOM is excluded from OLD matching and kept, unless every line is deleted, which leaves zero bytes. NEW lines use the first matched line's terminator, or LF when it has none. The last NEW line keeps the last matched line's terminator. Deleting trailing lines keeps the newly exposed line's existing terminator. Bytes outside matched lines are unchanged.

4B.5 Typed failure and lifecycle

The closed applicability codes are `EDIT_TARGET_MISSING`, `EDIT_NOT_UTF8`, `EDIT_SOURCE_UNSEEN`, `EDIT_SOURCE_CHANGED`, `EDIT_OLD_NOT_FOUND`, `EDIT_OLD_NOT_UNIQUE` and `EDIT_BLOCKS_OVERLAP` (`agent/edits.py:15–25`). An applicability failure ends the run with status FAILED and internal stop reason `EDIT_NOT_APPLICABLE`; ACP wire `stopReason` is `refusal` (`acp/spec.py:2212–2221`).

The fixed completion text is:

```
EDIT not applied to <path> (<code>): <reason>. Nothing was written.
```

During execution, the plan's WRITE terminal is `suppressed`, or `failed_no_effect` after the post-guard digest re-check. Every plan TEST is `suppressed` (`agent/runner.py:1482–1526`, `:1547–1552`). Binding failures occur before approval. Applicability failure stops the run; it does not enter fitness-driven replanning. Ordinary guard, I/O and cancellation failures keep their existing effect classification (`agent/runner.py:203–234`). Completion text is passed through by `acp/spec.py:2121–2122`.

4B.6 Logging and persistence

Typed applicability failures and successful application are offered to the opt-in ACP debug trace at `runner.py:edit`, hypothesis `P407-EDIT`. Digests are populated where available: a failure has no result digest, and a binding failure has no base digest. These EDIT debug records contain no file or model content (`agent/runner.py:2002–2012`).

`AgentRunResult.edit_failure_code` is set by `_edit_failure_result` to the specific closed `EDIT_*` applicability code for an `EDIT_NOT_APPLICABLE` run; it is `None` for other runner results. The seven-code vocabulary is enforced by the producing failure paths, not by the field's `str | None` type (`agent/models.py:175–177`; `agent/runner.py:1615–1639`; `agent/edits.py:15–52`).

Always-on `agent_run` telemetry now carries that code in the always-present nullable `edit_failure_code` key beside `stop_reason`; ordinary composition attempts to append it to `.optimus/telemetry.jsonl` (#412 / PR #414). This key carries no path, file or model content. Persistence is attempted, not guaranteed: sink errors are contained. After transport abandonment, ordinary `agent_run` emission is suppressed and no such persistence is attempted (`agent/runner.py:356–361`, `:1320–1350`; `telemetry/events.py:568–611`; `acp/bootstrap.py:129–156`; `telemetry/jsonl.py:34–43`; `telemetry/fanout.py:108–123`).

The ACP `agent/run` result payload also includes `edit_failure_code` because the dispatcher serializes the whole result model; this response field is not itself durable persistence (`acp/dispatcher.py:376–382`). Completion text still carries the code into transient conversation history, and the opt-in EDIT trace still carries it (`acp/spec.py:1589–1597`, `:2121–2122`; `agent/runner.py:2002–2012`). The code is not stored in Redis run metadata or the plan record: the Redis sink maps selected metadata fields and `AgentPlanRecord` has no such field (`telemetry/redis_sink.py:70–86`; `telemetry/redis_adapter.py:30–36`, `:134–146`; `agent/state_store.py:89–114`).

4B.7 Retained limits and ownership

Limit	Owning issue or retained contract
At most one declared WRITE/EDIT target; multi-target mutation directives cannot be expressed. This does not confine TEST subprocess effects	#408
Locating the target lines in large files	#409
Mechanical grounding of OLD	#258
No atomicity against external writers	#407 / PR #411 execution contract; `agent/runner.py`, `tools/mutation_tools.py`
Exact plan-text approval, without an ACP diff	#407 / PR #411 approval contract; `agent/runner.py`
WRITE remains a whole-file replacement, with line-ending/BOM/last-line fitting and mixed-ending refusal implemented at this pin; it does not use EDIT's localized byte-preservation contract	#410 / PR #413 implemented contract; `agent/directives.py`, `agent/tools.py`, `tools/line_endings.py`, `tools/mutation_tools.py` (§4B.1)

A mixed-ending WRITE raises `WriteRefused` before writing, settles WRITE as `failed_no_effect`, ends the run FAILED with `WRITE_LINE_ENDINGS_MIXED`, and suppresses every plan TEST; ACP forwards the refusal text and reports wire

`stopReason=end_turn` (`agent/runner.py:203–234`, `:1272–1297`, `:1455–1465`; `acp/spec.py:2123–2124`, `:2212–2221`). The refusal speaks only for that WRITE and its subsequent TESTs, not earlier planning-turn effects.

The guarded write declares `FILE_MUTATION` for WRITE and EDIT alike; EDIT is not assigned `PATCH_PROPOSAL`. Section 14 records the unit/runner, full-suite, CI and bounded live evidence with their original pins.

5. ADL parser continuity and architectural evolution

ADLParserStrategy retains parse_and_validate(adl_rules: list[str], proposed_diff: str) -> tuple[bool, list[dict]] as its abstract validation contract. PrototypeRegexADLParser retains the existing ASSERT(<a>) HAS NO DEPENDENCY ON <b> matching grammar, lowercases captured names for code import checks, and checks both an import <b> occurrence and the org.springframework.<b> substring in the diff. Violations retain rule, Structural Pollution metric and Violation: (<a> -> <b>) value. It returns true only when the violations list is empty. The regex implementation is a prototype and does not become a new semantic analyzer through this migration.

6. Resilient Aggregator Calling Layer and Tenacity Rules

All model completions route from the agent to the strict-loopback Gateway using the local shared secret. The Gateway resolves an Optimus model alias to an aggregator model identifier and calls the approved OpenAI-compatible upstream: OpenRouter by default (Vercel AI Gateway is a design option with no integration at 91fd64a).

The aggregator credential exists only in Gateway-owned local configuration or OS credential storage. There is no Vault, hosted Optimus account, or direct single-provider adapter.

Two agent-facing shapes, one upstream transport

- `/v1/responses` accepts the Responses API `input` field.
- `/v1/chat/completions` accepts the Chat Completions `messages` array.
- The validator rejects `messages` at `/v1/responses` and `input` at `/v1/chat/completions`.
- Both paths converge on one authenticated completion service.
- The surviving transport is `UrllibOpenAICompatibleClient` in `src/optimus_gateway/upstream_client.py` (built by `optimus_gateway.providers.build_upstream_client`).

The legacy host permits an initial attempt plus at most three retries (`optimus/retry/policy.py`, `max_retries = 3`; the enforced-policy limit is `MAX_ROUTE_ATTEMPTS = 2` in `optimus_model_policy/binding.py`). The inactive-policy Gateway separately permits at most three upstream attempts, including its legacy transient timeout/URL/5xx retries. Once enforcement is active, the Gateway permits one primary attempt and at most one identical-payload resend on the same route, only after a definitely NOT_SENT result (including a proven pre-connection timeout) or rejected HTTP 429. A timeout after connection, uncertain delivery, 5xx or unknown accounting is not resent. The host honors non-retryable classifications so it cannot multiply attempts. No alternative-model recovery is added. Each physical attempt retains its own receipt.

Provider-reported `usage.cost` is authoritative in the corrected architecture. Missing, null, negative, or malformed usage/cost fails closed before generated output is accepted.

6.1 OpenAI-compatible upstream pattern

```
class UrllibOpenAICompatibleClient:
    """Gateway-owned aggregator transport.

    The agent never receives the upstream URL or credential. The transport
    has no direct-provider selection branch.
    """

    def __init__(self, *, api_key: str, base_url: str) -> None:
        self._api_key = api_key
        self._base_url = base_url.rstrip("/")

    def create_message(
        self, *, model: str, input_text: str
    ) -> ProviderMessageResult:
        payload = {
            "model": model,
            "messages": [{"role": "user", "content": input_text}],
        }
        body = self._post_json("/chat/completions", payload)
        return parse_openai_compatible_message_and_usage(body)
```

`parse_openai_compatible_message_and_usage` must validate output, provider request identity, billing units, token/cache detail, resolved model/version when present, and provider-reported USD cost. The Gateway returns the requested agent-facing shape plus the same validated GatewayUsage contract.

```
Agent /v1/responses "input" --------\
                                     > completion service
Agent /v1/chat/completions "messages"/       |
                                             v
                        OpenAI-compatible aggregator transport
```

The upstream transport may use Chat Completions while the Gateway preserves both agent-facing contracts. Those shapes must never be mixed.

7. Composite Fitness Engine Scaffolding

```
class FitnessGateInterface(ABC):
    @abstractmethod
    def evaluate_gate(self, state: dict) -> Tuple[bool, dict]:
        pass

class DependencyRuleGate(FitnessGateInterface):
    def __init__(self, parser: ADLParserStrategy):
        self.parser = parser

    def evaluate_gate(self, state: dict) -> Tuple[bool, dict]:
        passed, violations = self.parser.parse_and_validate(
            state.get("adl_rules", []), state.get("generated_patch", "")
        )
        return passed, {"violations": violations}

class PlaceholderMetricGate(FitnessGateInterface):
    def __init__(self, max_complexity: int = 15):
        self.max_complexity = max_complexity

    def evaluate_gate(self, state: dict) -> Tuple[bool, dict]:
        return True, {
            "metrics": {
                "cyclomatic_complexity":  4,
                "maintainability_index": 82,
            }
        }

class PlaceholderTestArchGate(FitnessGateInterface):
    def evaluate_gate(self, state: dict) -> Tuple[bool, dict]:
        return True, {
            "testarch_status": "COMPLIANT",
            "graph_cyclical":  False,
        }

class CompositeFitnessEngine:
    def __init__(self, gates: list[FitnessGateInterface]):
        self.gates = gates

    def execute_harness_check(self, state: dict) -> dict:
        combined_telemetry = {}
        for gate in self.gates:
            passed, telemetry_delta = gate.evaluate_gate(state)
            combined_telemetry.update(telemetry_delta)
            if not passed:
                return {
                    "fitness_verdict": "FAIL",
                    "telemetry":       combined_telemetry,
                }
        return {
            "fitness_verdict": "PASS",
            "telemetry":       combined_telemetry,
        }
```

8. Patch Workspace Lifecycle Boundaries

Applying patches exposes systems to directory traversal attacks. Optimus normalises boundaries using strict commonpath comparisons and strictly isolates validation steps entirely inside a localised shadow workspace environment.

- **Shadow Workspace Environment**: Patches are written exclusively to a workspace-local shadow directory located at .optimus/shadow_env/. The patch diff is programmatically parsed, applied, and validated against the duplicate code tree via python file mutation loops.
- **Operating Mode Enforcement Contra**ct: If execution_mode equals AgentExecutionMode.PLAN, tool operations matching ToolOperationKind.WRITE or ToolOperationKind.EXTERNAL_MUTATION are completely bypassed and blocked from modifying local structures. Read-only inspection tools remain fully authorised.
- **Atomic Apply to Working Tree & Surfacing Rejec**tions: Modifications are applied to the working tree only after a clean PASS verdict, guaranteeing transaction atomicity and immediate rollback safety on any system fault. If a gate trips a FAIL signature, the patch is rejected, the transaction aborts, and the duplicate shadow environment is cleanly purged.

9. Tool Registry, Invocation Policy & Evidence Ledger

Tool use is policy-driven first, model-requested second: the model may request evidence, but the harness alone decides whether a tool is permitted, necessary, and cost-justified for the active execution mode and rigor tier. This section defines the typed tool registry, the deterministic invocation policy matrix, and the Evidence Ledger schema that records every external lookup alongside the Assumption Ledger.

A. Tool Class Enum & Registry Contract

```
class ToolClass(str, Enum):
    LOCAL_REPO_DISCOVERY         = "LOCAL_REPO_DISCOVERY"
    VALIDATION_GATE              = "VALIDATION_GATE"
    PATCH_WORKSPACE              = "PATCH_WORKSPACE"
    COST_TELEMETRY               = "COST_TELEMETRY"
    WEB_EVIDENCE                 = "WEB_EVIDENCE"
    PACKAGE_AND_ADVISORY_METADATA = "PACKAGE_AND_ADVISORY_METADATA"
    EXTERNAL_EXECUTION           = "EXTERNAL_EXECUTION"

class EvidenceReasonCode(str, Enum):
    API_DOCS_OUTDATED = "API_DOCS_OUTDATED"
    CURRENT_FACT      = "CURRENT_FACT"
    SECURITY_ADVISORY = "SECURITY_ADVISORY"
    USER_REQUESTED    = "USER_REQUESTED"
    PACKAGE_VERSION   = "PACKAGE_VERSION"

class SearchDepth(str, Enum):
    """Internal rigor presets. NOT assumed to equal the gateway's wire
    values 1:1 -- see TAVILY_SEARCH_DEPTH_MAP in section C, which is
    the only place an internal value is translated into the string
    actually sent to the provider via the gateway."""
    BASIC    = "basic"
    FAST     = "fast"
    ADVANCED = "advanced"

class ToolRegistryEntry(BaseModel):
    tool_name:            str
    tool_class:           ToolClass
    allowed_modes:        list[AgentExecutionMode]
    requires_policy_trigger: bool = False
    is_mutating:          bool = False
    max_calls_per_run:    Optional[int] = None

class ToolRegistry(BaseModel):
    entries:     dict[str, ToolRegistryEntry] = Field(default_factory=dict)
    # In-process call counters, keyed by f"{run_id}:{tool_name}". Single
    # process / single-instance only -- see the Redis-backed variant
    # below for concurrent or multi-process execution.
    call_counts: dict[str, int] = Field(default_factory=dict)

    def check_static_policy(
        self, entry: Optional[ToolRegistryEntry],
        mode: AgentExecutionMode, policy_triggered: bool,
    ) -> bool:
        if entry is None:
            return False
        if mode not in entry.allowed_modes:
            return False
        if entry.requires_policy_trigger and not policy_triggered:
            return False
        return True
```

```
    def is_allowed(
        self, tool_name: str, mode: AgentExecutionMode,
        policy_triggered: bool, run_id: str,
    ) -> bool:
        """READ-ONLY inspection (e.g. UI previews, dry-run explainers).
        Does not increment the call counter. Never use this to gate an
        actual tool invocation -- use authorize_and_record_call() below,
```

```
        entry = self.entries.get(tool_name)
        if not self.check_static_policy(entry, mode, policy_triggered):
            return False
        if entry.max_calls_per_run is not None:
            counter_key  = f"{run_id}:{tool_name}"
            calls_so_far = self.call_counts.get(counter_key, 0)
            if calls_so_far >= entry.max_calls_per_run:
                return False
        return True

    def authorize_and_record_call(
        self, tool_name: str, mode: AgentExecutionMode,
        policy_triggered: bool, run_id: str,
    ) -> bool:
        """The only method that should gate an actual tool call. Checks
        the static policy and the call-count ceiling, then increments
        the counter, as a single atomic step -- so authorization and
        bookkeeping can never drift apart, and a caller cannot forget
        to record a call it was just allowed to make."""
        entry = self.entries.get(tool_name)
        if not self.check_static_policy(entry, mode, policy_triggered):
            return False
        counter_key = f"{run_id}:{tool_name}"
        if entry.max_calls_per_run is not None:
            calls_so_far = self.call_counts.get(counter_key, 0)
            if calls_so_far >= entry.max_calls_per_run:
                return False
        self.call_counts[counter_key] = (
            self.call_counts.get(counter_key, 0) + 1
        )
        return True
```

The in-process dict above is sufficient for Phase 1's single-instance daemon, but is not safe across multiple worker processes and does not survive a restart mid-run. The production-grade path swaps the in-memory counter for a Redis-backed atomic increment, keyed identically and protected by a TTL matching the run's expected lifetime:

```
async def authorize_and_record_call_redis(
    registry: ToolRegistry, redis_client: aioredis.Redis,
    tool_name: str, mode: AgentExecutionMode,
    policy_triggered: bool, run_id: str, run_ttl_secs: int = 3600,
) -> bool:
    entry = registry.entries.get(tool_name)
    if not registry.check_static_policy(entry, mode, policy_triggered):
        return False
    if entry.max_calls_per_run is None:
        return True
    counter_key = f"toolcalls:{run_id}:{tool_name}"
    # INCR is atomic at the Redis level, so concurrent callers cannot
    # race past the ceiling the way two in-process checks could.
    new_count = await redis_client.incr(counter_key)
    if new_count == 1:
        await redis_client.expire(counter_key, run_ttl_secs)
    if new_count > entry.max_calls_per_run:
        await redis_client.decr(counter_key)
        return False
    return True
```

B. Deterministic Invocation Policy Matrix

The policy matrix maps task signals to permitted tool classes. The model never selects a tool class directly; it raises a typed signal and the harness resolves the matching policy row. PACKAGE_AND_ADVISORY_METADATA deliberately covers both package-registry lookups (PyPI, npm, Maven Central) and security-advisory lookups (OSV.dev, GitHub advisories): both are read-only, policy-triggered, and keyed by a package or CVE identifier, so a single tool class keeps the registry simple while the signal name (TASK_TOUCHES_DEPENDENCIES vs. TASK_TOUCHES_SECURITY_CVE) still makes the routing intent explicit.

9C. Typed Evidence Acquisition Wrappers

The agent authenticates only to the loopback Gateway. Search-provider mechanics remain behind `/v1/tools/web/search`; extract, package, and advisory capabilities have independent dependencies.

Strict-loopback settings

```
class OptimusGatewaySettings(BaseModel):
    gateway_url: str = "http://127.0.0.1:8765"
    optimus_api_key: SecretStr

    @model_validator(mode="after")
    def validate_loopback_gateway(self) -> "OptimusGatewaySettings":
        parsed = urlsplit(self.gateway_url)
        if parsed.scheme not in {"http", "https"}:
            raise ValueError("Gateway URL must use HTTP(S)")
        if parsed.username is not None or parsed.password is not None:
            raise ValueError("Gateway URL must not contain userinfo")
        if parsed.hostname not in {"127.0.0.1", "localhost", "::1"}:
            raise ValueError("Gateway URL must resolve to strict loopback")
        return self
```

There is no production mode, built-in hosted origin, extra-origin override, signed tenant profile, or non-loopback trust seam. The final source must remain aligned with the separately reviewed strict-loopback implementation.

9C.1 Deterministic search contract (target) and the implemented provider

Implemented at 91fd64a: the Gateway's `/v1/tools/web/search` route is served by `TavilyWebToolProvider` (`optimus_gateway/tool_providers.py`), configured with the Gateway-only `TAVILY_API_KEY`; its usage is recorded as provider-native `tavily_credits`. The contract below is the accepted target for replacing that provider with a dedicated aggregator request; it is not implemented in the Gateway.

The target converts an authorized search request into a dedicated OpenRouter Chat Completions request using one cheap Flash/Haiku-class model, low `max_tokens`, and `plugins: [{"id": "web"}]`. Search is never attached to the primary generation call.

```
{
  "model": "google/gemini-2.5-flash-lite",
  "messages": [{"role": "user", "content": "<minimal search request>"}],
  "max_tokens": 16,
  "plugins": [{
    "id": "web",
    "max_results": 3,
    "search_prompt": "<policy-bounded query>"
  }]
}
```

The Gateway would forward the effective domain policy, parse only `annotations[].url_citation`, independently revalidate every returned URL, and copy upstream usage/cost. Assistant prose and annotation character offsets are not evidence.

Measured baseline (architecture spike, 2026-07-27)

Measurement	Value	Scope
Minimal-call annotations present	3/3	`tools/run_openrouter_web_search_spike.py`, run against OpenRouter directly
Include-domain violations	0	same spike
Exclude-domain violations	0	same spike
Citations structurally complete	9/9	same spike
Mean latency per search	7,068.7 ms	same spike; one host, one day
Mean reported cost	$0.0051584	same spike; prices of that day

These values were produced by the spike tool, not by a Gateway test, and the spike is an experimental measurement harness, not production Gateway code. Three to five sequential searches would project to 21.2-35.3 seconds at that latency. The supplied Tavily comparison (about 1-2 seconds and $0.008/search) was not measured in this spike. The deterministic aggregator plugin is deprecated by OpenRouter (§9C.2), so the target itself is conditional.

9C.2 Plugin deprecation and successor gate

The deterministic OpenRouter plugin is deprecated. A live release probe must block release on removal or behavior change.

The first successor evaluation is a dedicated server-tool request with:

- `max_uses: 1`;
- a one-call budget;
- mandatory search-use accounting;
- valid structured annotations;
- domain-policy enforcement;
- fail-closed behavior when execution evidence is absent.

This is verified-or-fail, not deterministic by construction. If it fails, the fallback is a standalone search provider with a second credential and balance.

9C.3 Bounded direct extract

Extract performs a Gateway-side HTTPS fetch only for URLs recorded by a prior approved search in the same run. It:

- revalidates every redirect and final host;
- rejects URI userinfo and private, link-local, or loopback resolution;
- bounds connect/read time, bytes, redirects, and extracted characters;
- accepts only supported textual media;
- treats HTML and extracted text as untrusted input;
- never executes page content.

The spike fetched 270,008 bytes and parsed 60,077 characters in 589.5 ms. A 1,000,000-byte limit failed closed on a legitimate larger page; production limits and streaming behavior require explicit tests.

9C.4 Independent tool dependency construction

Status at 91fd64a: `optimus_gateway.providers.build_tool_dependencies` builds one `GatewayToolDependencies` bundle (web, package, advisory providers, tool policy, Redis tool state and the per-tool call cap) and returns `None` when the Tavily key, the tool Redis URL or the allowed-domain list is missing. The search configuration therefore still gates the package and advisory routes. That coupling must be removed regardless of the selected search backend. The separation below is that requirement: specified and not yet implemented.

```
@dataclass(frozen=True)
class ToolDependencies:
    search: SearchBackend | None
    extract: ExtractBackend
    package: PackageRegistryBackend
    advisory: AdvisoryBackend

def build_tool_dependencies(settings: GatewaySettings) -> ToolDependencies:
    return ToolDependencies(
        search=build_search_backend_if_configured(settings),
        extract=BoundedHttpExtract(...),
        package=PublicPackageRegistries(...),
        advisory=OsvAdvisoryClient(...),
    )
```

Route registration is capability-specific:

Capability	Availability rule (target)	At 91fd64a
Search	Selected backend configured and live acceptance gate passes	Tavily when `TAVILY_API_KEY`, tool Redis URL and allowed domains are set
Extract	Bounded HTTPS fetch and prior-search provenance support available	Same bundle; unavailable without the search configuration
Package lookup	Always available with public PyPI/npm/Maven clients	Same bundle; unavailable without the search configuration
Security advisory	Always available with public OSV client	Same bundle; unavailable without the search configuration

Do not delete the Tavily adapter until replacement acceptance tests and rollback review pass. Record that a standalone search backend may need to return if the aggregator search path is withdrawn. This finding is recorded in the correction map for assignment to an owning GitHub issue; this edition commissions no implementation.

Target contract after the named trusted-policy / Task 11 / coverage migration as applicable; the dated inactive production baseline is stated on the cover.

9D. Gateway Server-Side Policy Revalidation

The Gateway independently revalidates every privileged input. Agent-side checks are defense in depth and never authoritative.

Control	Gateway enforcement
Allowed domains	Intersect caller request with local allowlist; forward effective policy; reject returned URLs outside it
Extract provenance	Require exact prior search result for the same `run_id`; revalidate redirects and final URL
Capacity and accounting	Enforce the complete request guard under trusted policy; retain actual usage and explicit unknowns. Product monetary stops are removed only by the released Task 11 migration. Evaluation spend limits are separately scoped.
Call caps	Key by run and tool; do not share a paid-search configuration gate
Tool policy	Recheck tool class, policy signal, execution mode, and authenticated local subject
Usage	Reject missing, malformed, negative, or unparseable provider-reported cost

```
def authorize_tool_call(request, *, policy, ledger):
    effective_domains = policy.intersect_domains(request.domains)
    policy.require_tool_allowed(request.tool, request.execution_mode)
    policy.require_call_capacity(request.run_id, request.tool)
    require_final_capacity_and_output(request, policy)
    return AuthorizedToolCall(
        request=request,
        effective_domains=effective_domains,
    )
```

There is no org/project dimension. Cross-run spend policy remains `P9.85-FU-3` and is not added by this correction.

9E. Evidence Ledger Schema

Evidence entries retain structural provenance and provider-reported usage without adding a second normalized-credit charge.

```python
class EvidenceLedgerEntry(BaseModel):
    evidence_id: str
    run_id: str
    session_id: str
    request_id: str
    gateway_request_id: str
    provider_request_id: str | None
    provider: str
    model: str | None
    model_version: str | None
    cache_hit: bool
    billing_units: int
    cost_usd: Decimal
    source_url: AnyHttpUrl | None
    trust: Literal["trusted", "untrusted"]
    policy_reason: str
    recorded_at: datetime
```

`cost_usd` is copied from validated GatewayUsage. `billing_units` remains provider-native accounting normalized only to a stable integer field. Package/advisory operations may record zero billable units without inventing vendor charges.

Reconciliation methods are `total_cost_usd()` and `total_billing_units()`. Credit-named fields and `total_credits()` are removed under the separate USD rename.

9E.1 Protocol-visible USD rename (implementation complete; real-client evidence UNRUN)

The run-total field exposed in ACP result payloads for search, extract, package and advisory responses was a wire contract, so its rename from the credit-named `ledger_run_total_credits` was a contract migration, not an internal refactor.

Status at 91fd64a: the rename is implemented. `optimus/acp/dispatcher.py` emits `ledger_run_total_cost_usd` and `ledger_run_total_billing_units`; `EvidenceLedger` reconciles with `total_cost_usd()` and `total_billing_units()`; no credit-named ledger field remains under `src/`. The only remaining `credits` token is the provider-native unit label `tavily_credits` used for Tavily usage, which is accounting metadata, not a charge. Verified by code inspection and the unit and integration tests (`tests/unit/acp/test_dispatcher.py`, `tests/integration/evidence/test_mocked_evidence_flow.py`); the predecessor's real-client obligation is UNRUN because the `requires_acpx` tier was deselected in the sealed runs (Test Strategy §15.5, §16).

The migration's required evidence, with status at 91fd64a:

Record	Required evidence	Status
Search/extract response	`GatewayUsage` and run-total USD reconcile	Unit and integration tests
Package/advisory response	Free operation remains non-fabricated and the run total is stable	Unit and integration tests
EvidenceLedger	Decimal-safe sum by run/session/request identity	Unit tests
ACP schema	USD field documented; no compatibility shim for the credit-named field remains at 91fd64a	Code inspection
Real ACP client	Response parses without project-authored client assumptions	UNRUN (`requires_acpx` tier deselected in the sealed runs; Test Strategy §15.5)

The ledger remains append-only and rejects a duplicate charge for the same `gateway_request_id`. The rename changed no USD semantics and added no cross-run policy.

10. Usage Accounting Service and TimeSeries Policy

`UsageAccountingService` accepts only validated GatewayUsage/ProviderUsage records. It persists provider-reported billing units, tokens/cache details, model/version, and `cost_usd`.

```
# Illustrative shape. The implemented method is UsageAccountingService.record_gateway_usage
# in src/optimus/usage/accounting.py; it enforces the same validation before appending.
def record_usage(self, usage: GatewayUsage, *, context: RequestContext) -> None:
    require_nonempty(usage.gateway_request_id)
    require_nonnegative_decimal(usage.cost_usd)
    require_nonnegative_int(usage.billing_units)
    self.ledger.append(
        ProviderUsage.from_gateway_usage(usage, context=context)
    )
```

It never substitutes a locally calculated charge when provider cost exists. Missing, null, negative, or malformed cost fails closed before generated content or evidence is applied.

A versioned price snapshot may remain as labeled diagnostic/comparison metadata only. It cannot overwrite provider-reported cost or silently supply a release-grade value.

Required identity

- `run_id`
- `session_id`
- `request_id`
- `gateway_request_id`
- `provider_request_id` when returned
- aggregator and actual provider when returned
- requested alias and resolved model/version

10.1 RedisTimeSeries persistence

RedisTimeSeries retention and idempotent creation mechanics remain unchanged.

```
TS.CREATE optimus:usage:<run_id>:cost_usd
  RETENTION 2592000000
  LABELS run_id <run_id> metric cost_usd

TS.CREATE optimus:usage:<run_id>:billing_units
  RETENTION 2592000000
  LABELS run_id <run_id> metric billing_units
```

Use `TS.ALTER` when an existing series needs required labels or retention correction. Never drop existing measurements during schema alignment.

Reconciliation

For each accepted Gateway request:

```
Gateway response cost_usd
  == ProviderUsage.cost_usd
  == EvidenceLedger entry cost_usd, when evidence-producing
  == RedisTimeSeries appended value
```

Duplicate `gateway_request_id` records must be idempotent or rejected before a second charge is written. Price snapshots are diagnostic only. Observability export is not a billable usage record.

10A. Provider Usage Ledger and Observability Export

ProviderUsage persists the wire-level GatewayUsage fields verbatim and adds reconciliation attribution: aggregator, actual provider, resolved model/version, service, native unit, run/session/request IDs, and an optional diagnostic price-snapshot ID.

It does not add an Optimus credit charge. `cost_usd` is the upstream-reported aggregator charge. Public package/advisory calls record operational evidence without invented cost.

OpenTelemetry export

The agent sends authenticated structured trace events to `/v1/observability/traces`. The Gateway:

1. validates the event schema and local subject;
2. redacts credentials and sensitive content;
3. maps fields to OTel spans/events;
4. exports OTLP;
5. records delivery status and trace identifiers.

Phoenix is the documented local default. Required attributes include run, session, request, Gateway/provider request, execution mode, generation scope, model/provider, cache, `cost_usd`, billing units, policy/tool, validation, retry, and failure fields.

Trace delivery has no allocated or amortized per-request observability charge.

11. Sprint 1 Implementation Checklist

Trust boundary

- Accept `127.0.0.1`, `localhost`, and `::1`; reject every non-loopback Gateway URL.
- Remove production-mode, extra-origin, hosted-origin, and tenant-profile bypasses.
- Prove WSL2 agent and Gateway run in the same network namespace.
- Prove the agent process resolves only Gateway URL/shared secret.
- Prove the Gateway credential never appears in agent state, logs, errors, telemetry, or children.

Model transport

- Use OpenRouter as default aggregator and the OpenAI-compatible transport.
- Retain Vercel only if Python integration is modest; otherwise assign a named backlog entry. (Status at 91fd64a: no Vercel integration exists.)
- Remove direct provider branches and `anthropic_client.py`. (Done: no `anthropic_client` module at 91fd64a.)
- Parse provider-reported usage/cost; fail closed when incomplete.
- The legacy host permits an initial attempt plus at most three retries. The inactive-policy Gateway separately permits at most three upstream attempts, including its legacy transient timeout/URL/5xx retries. Once enforcement is active, the Gateway permits one primary attempt and at most one identical-payload resend on the same route, only after a definitely NOT_SENT result (including a proven pre-connection timeout) or rejected HTTP 429. A timeout after connection, uncertain delivery, 5xx or unknown accounting is not resent. The host honors non-retryable classifications so it cannot multiply attempts. No alternative-model recovery is added. Each physical attempt retains its own receipt.

Tool capability

- Separate package/OSV route construction from paid-search configuration. (Open at 91fd64a: one bundle, gated by `TAVILY_API_KEY`, tool Redis URL and allowed domains.)
- Keep Tavily until replacement acceptance and rollback review pass. (Tavily is the implemented provider at 91fd64a.)
- Prove deterministic annotations, domains, URL revalidation, usage, and plugin live gate. (Spike evidence only, 2026-07-27; no Gateway implementation of the plugin path.)
- Prove extract redirect, SSRF, media, size, time, and provenance controls.

11.1 Accounting, telemetry, and release evidence

Accounting

- Persist provider-reported Decimal-safe `cost_usd` and billing units.
- Preserve run/session/request/Gateway/provider request attribution.
- Remove credit-named fields in the separate wire-aware USD migration.
- Prove RedisTimeSeries retention and idempotent creation/alter behavior.

Telemetry

- Export real OTLP spans to a live Phoenix evidence tier.
- Prove no LangSmith dependency, key, or direct backend egress.
- Record validation, retry, tool-policy, and final-disposition attributes.

Release gate

The agent process has only Gateway URL/shared secret. The Gateway process has only the approved aggregator key after migration acceptance. No direct-provider, Tavily, or LangSmith key remains.

Real evidence tiers must use their named dependencies. ACP protocol evidence uses the independent `acpx` client. Fake-based tests remain unit evidence and cannot justify live sign-off.

The deterministic search plugin is release-blocking while it remains the Phase 1 path. A successor server tool is not accepted until live verified-or-fail evidence proves exactly one search, search-use accounting, valid annotations, domain controls, and fail-closed behavior.

Target contract after the named trusted-policy / Task 11 / coverage migration as applicable; the dated inactive production baseline is stated on the cover.

11A. Test Coverage and Observability Cross-Reference

The Test Strategy v1.9 is authoritative for coverage, release, live-dependency, and trace evidence.

Code coverage gates use the Optimus product group and separate context_engine / optimus_model_policy 80% floors; all-source aggregate is informational, with safety-critical modules protected from regression. OTel/OTLP trace assertions validate debugging and regression fields but do not count toward code coverage.

Evidence class	Required dependency
Unit	Fakes allowed; no network/I/O unless intrinsic
Redis integration	Real TimeSeries-capable Redis
Gateway live	Real Optimus credentials and live local Gateway
ACP protocol	Independent `acpx` client
Trace live	Real OTLP export to Phoenix
Release	Agent/Gateway credential and egress scopes proven separately

LangSmith trace assertions and amortized observability accounting are deleted.

Target contract after the named trusted-policy / Task 11 / coverage migration as applicable; the dated inactive production baseline is stated on the cover.

12. Guardrail and Workflow Component Contracts

All model-touching guardrails use the same loopback Gateway, developer aggregator account, actual accounting and explicit evaluation caps, provider-reported cost, and OTel trace path.

This section specifies the Phase 1 enforcement and workflow components introduced as a cross-cutting concern in HLD §13. The authoritative policy rationale lives in the companion Agent Execution Guardrails & Workflow Strategy v1.4 (§2-§9); the representative Pydantic type shapes are in its §10.2. The contracts below are owned by this section. All model-touching elements (the borderline classifier and the loop completion evaluator) route through the Optimus Gateway under the existing developer aggregator account and normalized ledger (§0A, §10A); the guardrails introduce no second cost path.

12A. Permission & Pre-Tool Enforcement

PermissionPolicy evaluates mode -> user_deny -> project_allow -> impact -> classifier and returns a single PermissionDecision (ALLOW | DENY | HOLD); deny always precedes allow; the classifier cannot overturn a deny.

PermissionDecision contains verdict, layer, rule_id, reason, and requires_human_approval.

PreToolGuard fires after the agent assembles a tool call, before execution (cf. PreToolUse), for every tool class (bash, file_edit, mcp_call, web); deterministic checks run before any classifier. It returns PreToolResult.

CommandSafetyValidator is a local, in-process shell validator covering destructive commands, pipe-to-shell, environment/credential access, Unicode homoglyph/confusable characters, ANSI/control sequences, insecure transport, and unexpected network egress. Satisfiable by Tirith or an equivalent validator; the design takes no hard dependency on any one implementation. Implemented in-process in `optimus/guardrails/command_safety.py` at 91fd64a; Tirith is not a dependency.

ToolInvocationAuditEvent is an append-only record of each decision (verdict, layer, failed checks, approver) feeding the same trace sink as evidence and cost telemetry.

12B. Prompt-Injection & MCP Supply-Chain Trust

MCPTrustRegistry captures server_id, manifest_hash, allowed_tools, permission_scope, and approved. MCP servers are never auto-loaded from cloned repositories; a manifest-hash change forces re-approval; tool descriptors (name/description/schema) are inspected for injection before being surfaced to the planner.

ConfigTrustScanner treats agent config and rule files as code, scanning on ingest for embedded instructions, exfiltration endpoints, and homoglyph/ANSI content before they may influence behavior.

12C. Bounded Agent Loops

GoalLoopController, IterationState, CompletionEvaluator, ProgressLedger, LoopBudgetPolicy (max_iterations, max_wall_clock_minutes, repeated_failure_limit; `max_budget_usd` defaults to None and is set only by an explicitly capped evaluation caller; the model default for repeated_failure_limit is 3 in `optimus/loops/models.py` and ACP planning composes 2), and LoopStopReason (COMPLETED | MAX_ITERATIONS | WALL_CLOCK | REPEATED_FAILURE | HUMAN_HALT; BUDGET_EXHAUSTED only for independent evaluation) are the bounded-loop contracts.

Persistent state lives in files, git history, task manifests, traces, and the evidence ledger (§9E), not in an ever-growing chat context. The pre-tool guard (§12A) is never bypassed inside a loop, and the completion evaluator is a reserved cheap Gateway-routed capability. This section does not claim it is active; unknown costs do not latch a product goal loop. Task 11 removes the product monetary predicates on the reviewed cf9bb9d branch. Independent evaluation monetary policy remains explicit.

12D. Curated Workflow Skills

`SkillRegistry`, `SkillManifest` (`name`, `description`, `globs`, `allowed_tools`, `owner`, `version`, `trust_level`), `SkillTrustPolicy` (draft/untrusted skills blocked in Agent mode), and `SkillInvocationPolicy` define curated workflow skill resolution.

A skill's declared `allowed_tools` are enforced by the pre-tool guard (§12A) - a skill can never widen the agent's tool surface - and a skill can never override project or user deny rules.

13. Context Engine, policy and cost integration contracts

Edition scope. Registry admission, enforcement, production construction and Auto statements explicitly refreshed below describe `bf440cb`. Unrefreshed Plan 12.2 material retains its prior provenance and is read with the controlling 2026-10-18 ordinary shipping amendment (`docs/sources/plan-12-2-ordinary-shipping/architecture-amendment.md`), including compaction, numeric policy, profiles, maintenance and historical evidence. The new §13.10 specifies Auto. ADR-024 controls interactive loop behavior; retained interim limits and legacy goal-loop descriptions do not override it.

13.1 Source-to-view boundary

Canonical `ConversationTurn` records remain host-owned and sanitized. `make_history_snapshot` in `optimus.context.adapter` creates immutable ordinary turns and protected host projections from committed records. Supply the current sanitized prompt separately. `HistoryRevision` binds session, committed generation, last sequence and source digest. No summary changes canonical records or manufactures effects/approval facts.

`ContextEngine.prepare_view(snapshot, *, strategy, parameters, limits, checkpoint, maintenance, cancelled)` returns a bounded `PreparedView`. `StrategyParameters` carries compaction tail, hybrid anchor/tail, summary output cap, max_maintenance_calls, prompt_version and format_version; all fields participate in its digest. `ViewLimits` carries explicit source, history/maintenance token and transient byte bounds, including required `summary_max_bytes` and optional `maintenance_calls_remaining` (None for standalone use, a finite nonnegative integer supplied by the host; booleans rejected). A valid summary byte bound is at least one and strictly below the transient bound. No deployment values are inferred from dataclass defaults.

The injected estimator is subadditive over any strings the engine concatenates: `estimate(a + b) <= estimate(a) + estimate(b)`. Production policy uses the admitted route's conservative `ceil(r * UTF8_bytes)` profile with non-negative overhead. At `bf440cb`, ADR-025 also permits an explicitly recorded per-entry empirical admission whose estimator remains `verified: false`; its assumptions and per-reply checks do not establish a universal token or framing bound. The complete engine-assembled text is checked before each callback. ViewLimits.maintenance_input_tokens=131072 bounds that text only; the host summary prompt and wire framing are additional, and the complete Gateway request plus reserved output has its own route-capacity check. A downloaded tokenizer or fixture ratio does not silently become a new registry estimator mechanism.

Released offline numeric set (verified route still required)

Where these values live at 91fd64a: the reviewed test profiles and test fixtures (`optimus_model_policy/test_profile_qualification.yaml` output reserves 32768/8192; `tests/` fixtures), not shipped production defaults. `MAX_CONTEXT_CEILING_TOKENS = 262_144` (`optimus_model_policy/registry.py`) and `context_ceiling_tokens: 262144` (`defaults.yaml`) are the ceiling constants; `CONVERSATION_MAX_BYTES = 524_288` (`optimus/acp/conversation.py`) is the absent-engine source floor.

Field	Proposed value / unit
Absent / attached source	524288 / 1048576 UTF-8 bytes
Prompt-time record reservation	131072 bytes; not a reply cap
Transient engine input	524288 bytes
History allowance	min(131072 tokens, floor(0.8 * usable_input)); protected state first
Engine maintenance text	131072 estimated tokens; host prompt/wire framing extra
Summary output / bytes	S <=8192 tokens; summary_max_bytes=floor(S/r)
Compaction tail	32768 tokens
Hybrid anchor / tail	16384 / 65536 tokens, deduplicate overlap
Implementer output	32768 tokens, enforced actual route cap
Effective total	min(262144, verified window), includes output once

Field	Proposed value / unit
Maintenance / repacks	18 shared logical callbacks; at most one repack after initial view

Compaction summarizes outside its exact whole-turn tail; 131072 is a history ceiling, not a delayed trigger. At verified r=1 these budgets behave largely as byte budgets, less estimator/framing overhead; exact counting is deferred. The named D7 exception is accepted (ADR-018, operator source S4): keep the 524288-byte source floor/warning but refuse an over-capacity complete request readably with OPEN disposition, on absent-engine and attached full-history fallback. No provider guard exemption.

13.2 Whole-turn maintenance and checkpoints

Pack complete rendered turns against both conservative per-piece token sums and exact UTF-8 bytes. Count every separator and header. Reserve actual bytes/tokens for a reused prior summary; reserve the effective summary token cap and maximum summary bytes for each future rolling summary. Admit equality, close a chunk before either bound is exceeded, and preflight the entire finite chunk plan before the first callback. If even one required whole turn cannot fit, fail without starting maintenance for that plan.

Fresh and reused summaries pass the same token/byte/format validation. Rebuild an over-bound checkpoint from canonical source; a rejected rebuild publishes nothing. For supported byte-ratio profiles derive `S = min(summary_output_tokens, maintenance_output_tokens)` and `summary_max_bytes = floor(S / r)` with exact arithmetic. Invalid field relationships are configuration conflicts, not reasons to increase admission silently.

Checkpoints contain host-issued covered turn IDs/digests, strategy/parameter/prompt/format identity and source revision. Reuse requires an unchanged covered prefix, checked turn by turn. Cancellation, stale revisions, changed source/configuration, malformed summaries and partial results never publish a checkpoint. Coverage metadata is created by the host, not trusted from model prose.

Typed unavailable/unsupported maintenance status maps to `maintenance unavailable`; actual failure and rejected output retain attempt receipts and their own failure reasons. The filed C1 message map, common OPEN ending, fallback notice and capacity-recovery wording remain exact. A maintenance cancellation can still yield the older internal `maintenance_failed` diagnostic while the host returns cancelled; record this nonblocking residual without commissioning a diagnostic-code change in C1/C2.

13.3 Host attachment and final request

`ContextAttachment` in `optimus.context.assembly` is frozen and requires the engine, parameters, limits, captured source/route identity, receipt sink and finite max_repacks. The released offline setting is one repack after initial packing. Keep StrategyParameters and its checkpoint digest unchanged across the turn. Pass ViewLimits.maintenance_calls_remaining = max(0, 18 - callbacks_already_consumed) for initial preparation and each repack; this execution-state field is outside summary identity. The engine preflights the complete chunk plan against min(parameters.max_maintenance_calls, remaining) before any callback. Zero remaining permits a valid zero-call reused/sliding view and refuses a plan needing calls. Reuse still requires matching source, coverage, estimator and token/byte bounds. Retain the host defensive counter and allowance classification; repack checkpoints remain unpublished.

At `bf440cb`, normal production bootstrap is implemented under active trusted registry enforcement. `build_configured_server` uses `trusted_snapshot()` and `ordinary_route_policy`, then `ordinary_attachment` when an eligible summarizer exists (`src/optimus/acp/bootstrap.py:267–320`; `src/optimus/acp/context_attachment.py:51–103`). `ordinary_attachment` constructs the plain ContextEngine and Gateway-backed maintenance; without an eligible summarizer it returns None. This is production construction, not a test-only constructor.

The ordinary default planner is the cheapest eligible medium model, otherwise the cheapest eligible easy model (`src/optimus/agent/defaults.py:19–29`). In the shipped ordinary policy that is Haiku 5.5; the former claude-haiku-4.5 alias is not the enforced default. Auto uses its separate composition and per-turn binding (§13.10). Historical test-profile evidence and approval identities stay attached to their named profiles; an ordinary or Auto launch does not inherit their authority. The amendment controls unrefreshed Plan 12.2 behavior and evidence; no historical pass is promoted to this pin.

An `AttachedTurn` captures history/revision, mode/strategy/parameters, route/model/registry and request identity before any interleaving await. Setters affect later turns. It exposes separate `current_prompt`, exact `selection_text` and inert `conversation_envelope`. Do not hold a configuration lock across provider calls or approval.

`probe_floor` measures full committed canonical history plus provisional prompt without changing disposition. If the engine is unavailable and that projection is <=524288 bytes, prepare full-history fallback with the filed notice and still apply the final complete-request capacity guard. Above the floor, refuse before answer/planning dispatch and keep the thread OPEN. Attached storage class persists through health faults; repeated faults never latch CAP_CLOSED.

Bind the admitted request/view digest to the stored plan. Approved application reuses it and the exact artifact hash; it neither summarizes again nor reselects a model. A changed target requires existing freshness/replan/approval handling. Projection of an old approval never approves a revised plan.

13.4 Registry and final Gateway enforcement

Strict YAML rejects duplicate/non-string keys, aliases, anchors, merge keys and custom tags. Packaged defaults plus operator overrides yield one immutable, canonically hashed `RegistrySnapshot`. Formatting changes alone do not alter its effective identity. Neutral policy code imports no host runtime. Trusted launch composition supplies the actual validated object to host and Gateway; a caller-supplied hash is not authority.

At `bf440cb`, eligibility requires explicit role assignment, capabilities, recorded endpoint/quantization and verified window/output facts, supported reasoning controls, the applicable reserve and any required summary receipt/disclosure. The window floor is 262144 for non-classifier roles and 8192 for classifier; the classifier remains subject to the validator's positive reserve requirement even though the decisions endpoint reserves no output (`src/optimus_model_policy/registry.py:98–117`; `src/optimus_model_policy/validation.py:80–170`).

ADR-025 permits two independent admission parts on each model entry: `empirical_estimate` admits its explicitly unverified empirical estimator; `provider_family` admits its recorded provider base slug while requiring the reply's provider name. Neither relaxes the other part's checks. `conservative-utf8` uses ratio 1, 16 tokens per message and 256 fixed, `verified: false`. Evidence, assumptions and limitations participate in the effective registry identity. A first call can exceed the estimate or be charged before reply validation stops later calls to that model.

The operator override is the regular file `model-registry.yaml` in the trusted operator config root. Ordinary composition applies it over defaults; Auto applies it over defaults plus the Auto overlay. Launch approval binds the resulting effective hash, and host/Gateway compose independently; test profiles never read the override (`src/optimus_model_policy/binding.py:111–170`; `src/optimus/acp/launch_gate.py:701–728`). Unknown facts outside those explicit admissions and an unknown trusted snapshot do not authorize upstream dispatch.

For completion replies on exception routes, retain usage/finish validation, reported input at most the estimate where empirically admitted, recorded-provider equality where family-admitted, and one router attempt. An empty pipeline passes; a nonempty pipeline passes only ADR-026's exact shape: one `guardrail`, one closed-facts record, `flagged` false, `detected` false or missing, `decision`/`action`/`guardrail_scope` missing and `verdict_conflict` false. Explicit nulls, wrong types, extra stages or other facts refuse. Acceptance proves neither unchanged content nor data handling. A failed check stops only that model for the Gateway's remaining life; stops cannot be cleared (`src/optimus_gateway/model_policy.py:87–101`, `:224–288`). Decisions have the distinct reply contract in §13.10 because System One does not supply completion finish/router facts. Fixing model aliases does not permit direct provider keys. The legacy host permits an initial attempt plus at most three retries. The inactive-policy Gateway separately permits at most three upstream attempts, including its legacy transient timeout/URL/5xx retries. Once enforcement is active, the Gateway permits one primary attempt and at most one identical-payload resend on the same route, only after a definitely NOT_SENT result (including a proven pre-connection timeout) or rejected HTTP 429. A timeout after connection, uncertain delivery, 5xx or unknown accounting is not resent. The host honors non-retryable classifications so it cannot multiply attempts. No alternative-model recovery is added. Each physical attempt retains its own receipt.

At the final provider boundary enforce `complete_input_estimate + output_reserve <= min(262144, verified_route_window)`, `0 < output_reserve <= verified_route_max_output`, and the exact upstream cap/finish semantics. Count all messages, framing, tools/files/evidence and reasoning reservations; count output once. A complete-request tokenizer may inform route verification, but production use requires a registered supported and reviewed mechanism; neither sample ratios nor broad tokenizer-family labels suffice.

A WRITE- or EDIT-bearing result is usable only with a verified complete successful finish. Length-limit, missing or unknown finish fails before parse/store/approval/execution, even if the prefix parses. Chat incomplete output is visibly incomplete. Preserve receipts for rejected/discarded/cancelled work.

13.5 Exactly-once settlement and D6

`StageReceipt` identifies session, turn, stage, attempt, available Gateway/provider request IDs, requested/resolved model/provider, captured configuration/registry, cost or explicit unknown, token/cache data and time. Turn/scope summaries retain known subtotal, unknown IDs, receipt identities and completeness. Duplicate identical receipts are idempotent; conflicting duplicates are integrity errors. Actual attempts count once even when their summary/view/result is stale or discarded. Applying a stored plan never charges its old planning again.

Strict Gateway usage validation remains: invalid cost is not a valid zero-cost `ProviderUsage`. Record the attempted stage as unknown and refuse its unverified successful result as required; keep known subtotals before/after it. Unknown accounting does not introduce a session-wide product monetary stop or automatic retry. Telemetry must not debit an `agent_run` projection a second time.

Configured per-turn/session alerts emit once per actual threshold crossing, with stable scope/crossing identity. This draft invents no operator alert amount. Daily totals require durable reconciliation and explicit IANA timezone/day identity; absent prerequisites make that scope unavailable. P9.85-FU-3 and P11.26-CAND-2 retain existing ownership. Operational public requests and trace export do not acquire fabricated provider charges.

13.6 Product/evaluation policy migration

The reviewed Task 11 implementation removes product dollar predicates from Agent planner/runner and non-ACP goal-loop paths, positive-dollar construction requirements and stop-bearing telemetry/planner prompts. Use an explicit optional evaluation monetary policy only for independently capped callers. Keep product accounting and count/time/failure/cancellation/security/capacity controls. No renamed limit, hidden maintenance subtraction or large/infinite sentinel is acceptable.

The three strict xfails at dea4624 were held cost-stop-removal evidence. Task 11 now satisfies those assertions at cf9bb9d and removes their markers; the final full runs verify the product/evaluation split, and the change was merged to `main` via PR #276 (`c509fc8`). A result costing more than the former $0.05 must still be delivered successfully on a product path; an independently capped evaluation must stop under its separate envelope. Preserve the current three planning rounds, 30-minute limit, repeated-failure limit two and single-call Chat.

13.7 ACP projection and meter

Publish one full-set configOptions projection that preserves model/mode/strategy options and pinned option/setter IDs. Show the strategy picker only when configured attached, retaining it through engine faults. Capture each turn's configuration before awaits; use one revision/resync mechanism so an old publication cannot clear newer pending state. Sliding's filed omission disclosure appears on the first admitted sliding turn and reports actual delivery.

The meter records at local Gateway dispatch before its response, using the largest dispatched planning/answer input with that request's usable capacity, choosing smaller capacity on ties. Maintenance cost contributes to cost records, not the ring. A host capacity refusal yields no fabricated reading. A non-capacity Gateway preflight rejection can leave a reading for a request sent locally but never sent upstream; this meter is not proof of provider delivery. Distinguish storage warnings, strategy-unavailable fallback and whole-request capacity refusal.

13.8 Effect proof and rollout

Maintain Task 2's real WRITE/TEST registration, start gates, terminal evidence and cancellation behavior. At `57e2069`, EDIT registers and settles as the plan's WRITE operation, plus its read-before-write READ. During execution, `EDIT_NOT_APPLICABLE` settles WRITE as `suppressed`, or `failed_no_effect` after the post-guard digest re-check; every TEST is `suppressed`. Binding failure is before approval. Applicability failure stops the run without fitness-driven replanning (§4B). A normally returned TEST is executed even when its test exit is nonzero; test verdict and operation terminal are distinct.

Denied/not-started work is reflected in a fresh consistent effect snapshot before commit. Already-started operations are not promised to be interrupted. Never reconstruct authority from mutation counts or model prose.

Verify fresh-session rollout on the actual production base. If durable replay/resume arrives before rollout, its owner must supply reviewed provenance before old placeholder effects are treated as authoritative. Keep the sandbox's unsynchronized repair gap disclosed until its separately authorized synchronization. The migration adds no durable-history mechanism.

13.9 Implemented capacity outcomes and test launch

Once trusted enforcement is active, only INPUT_EXCEEDS_CAPACITY selects the absent-engine capacity text; CAPACITY_REFUSED without a known reason stays generic. Preserve known costs from earlier requests in the turn. The rejected request itself has zero upstream attempts.

Chat could not answer because this request exceeds the model's input capacity. This request was not sent to a model. A shorter prompt or a narrower request involving fewer workspace files may help. If earlier conversation history is the cause, start a new thread. This thread stays open.

Planning stopped because its next request exceeds the model's input capacity. That request was not sent to a model. A shorter prompt or a narrower request involving fewer workspace files may help. If earlier conversation history is the cause, start a new thread. This thread stays open.

The request meter uses complete packed input/usable capacity from the final policy estimator. Warn at 80% once per session; a jump over capacity emits refusal directly. Storage’s 80% notice remains separately labelled storage.

The source-owned qualification/attached profiles are selected by name only. HMAC launch authorization binds the security snapshot containing the effective profile hash; host and Gateway independently compose it, and the authenticated child manifest binds the same identity. The host literal check is internal consistency, not a separate operator approval. A profiled host must spawn its own profiled Gateway. Already-listening/external Gateway, no child, --no-auto-start, --framed and --strict are refused; a mismatched explicit model refuses before Redis/Gateway/server. No run-gateway profile flag is needed. That statement concerns the named test profiles; Auto's manually managed Gateway uses the separate `run-gateway --auto` option below.

13.10 Auto composition and per-turn routing

13.10.1 Selection and launch authority

`auto` is a composition selector, never an upstream model ID. Only `OPTIMUS_AGENT_MODEL=auto` selects it; `--model auto` and Auto combined with a different explicit `--model` refuse (`src/optimus/agent/defaults.py:32–42`, `:49–70`). Unsetting the variable, or selecting an eligible exact model ID, returns to ordinary fixed-model selection.

`auto_snapshot()` composes packaged defaults, `auto_overlay.yaml`, then the trusted operator override. `trusted_snapshot()` remains ordinary composition. The security literals bind both `_model_registry` and `_composition=auto`; an ordinary approval cannot authorize Auto, or conversely, even if hashes coincide. Auto and a named test profile are mutually exclusive (`src/optimus_model_policy/binding.py:73–84`, `:143–175`; `src/optimus/acp/launch_gate.py:701–728`).

After launch authorization, `compose_auto_composition` recomposes and compares the approved literal and composition. Startup validates the default planner and classifier before services start. A missing eligible classifier or one requiring Contributor disclosure refuses; the decisions route cannot verify that disclosure (`src/optimus/acp/auto_composition.py:212–236`; `src/optimus/acp/__main__.py:592–601`).

The auto-started Gateway receives `--auto-composition` and any trusted override path, composes independently and verifies the signed registry binding. A manually managed Gateway uses `optimus-trust --workspace-root <workspace> run-gateway --auto`. That command requires a durable workspace approval binding the registry literal and composition; it cannot authorize a changed override merely by signing it (`src/optimus/acp/local_infra.py:523–536`; `src/optimus_gateway/__main__.py:107–170`; `src/optimus/acp/launch_approval_cli.py:1068–1101`, `:1150–1155`).

The shipped overlay's effective composition, without an operator override, is the approved `f11dc3ab…`: easy Luna then Haiku; medium/complex/escalation Gemini 3.8 Flash; classifier Jev 1.14; summarizer Haiku 5.5. Price ordering is computed from eligible assignments, not YAML list order. Model IDs, prices, endpoints and admission evidence are configuration; no per-model task-success guarantee follows. Ordinary roles do not gain the overlay. The approved overlay retains historical ADR-027 evidence text inside its hash; its decision record is now ADR-028.

13.10.2 Frozen input and H

Before classification awaits, the adapter captures the sanitized prompt, turn mode, attachment metadata and last committed routed tier. `ClassifierInput` (`auto-input-v1`) serializes a prompt excerpt of at most 4096 UTF-8 bytes with a truncation flag, CHAT/AGENT mode, attachment count, size bucket, up to five sorted extensions and previous tier. It carries neither full history nor attachment contents. The previous tier is published from committed completed/rejected routed turns, not failed or cancelled work (`src/optimus/agent/auto_contract.py:26–67`, `:86–122`; `src/optimus/acp/spec.py:1787–1807`, `:2144–2148`).

`HybridClassifier` implements H, accepted by ADR-028:

- score the frozen input using the pilot's rules and cut points 0.5 and 4.5;
- at rule confidence >=0.8, use the rules' answer without a classifier call;
- otherwise ask the configured classifier once using the frozen `choice` question;
- apply the final confidence threshold 0.95 to a valid answer: below it, move up one tier, capped at complex.

Confidence must be a finite non-boolean number in [0,1]. Missing/malformed confidence is abstention, not a confident answer. Abstention, invalid answer, classifier timeout/error or a stopped classifier yields no usable tier; resolution starts at complex. Accounting-integrity failure and cancellation do not take that routing fallback (`src/optimus/acp/auto_hybrid.py:52–64`, `:104–130`, `:157–186`; `src/optimus/acp/spec.py:1805–1841`).

The host fits the complete typed decision request to its bound by shrinking only the prompt excerpt; if no excerpt fits, no decision is sent (`src/optimus/acp/auto_hybrid.py:188–203`). The thresholds are the accepted mechanism, not calibrated probabilities or proof of task suitability.

13.10.3 Resolver, binding and stops

`AutoRouter` is constructed once per launch with the approved snapshot, eligible route policies, per-model context limits, classifier and one monotonically growing host `StopSet`. `resolve` selects the cheapest eligible unstopped model of the requested task tier. It falls through stopped models within that role, then unavailable tiers upward, then tries the unstopped default planner. The last fallback can be an easy or medium model. With none available it raises `AUTO_NO_ROUTABLE_MODEL`; the adapter refuses recoverably before any planning/answer call (`src/optimus/acp/auto_composition.py:239–300`).

The adapter captures the chosen route binder and rebinds the attached turn's limits before planning/answer work. The selected planning/answer model remains fixed through that turn and approved application; applying an old plan does not classify it again. The summarizer remains the separately eligible summarizer, not whichever implementer H selected (`src/optimus/acp/auto_composition.py:303–362`; `src/optimus/acp/spec.py:1844–1851`; `tests/unit/acp/test_auto_turn_routing.py:174`).

Every Auto runner and summarizer shares the launch's host stop set with the resolver. Proven exception-route stops are retained even if receipt reporting raises; the Gateway independently keeps permanent per-model stops for its process life. Ordinary failures do not automatically stop all models. No stop is cleared as a retry-to-green mechanism (`src/optimus/agent/runner.py:1067–1078`; `src/optimus/context/maintenance.py:291–320`; `src/optimus_gateway/model_policy.py:87–101`).

These confidence/availability route-ups happen before planning/answer dispatch. They do not switch models mid-turn or implement ADR-009 struggle escalation. The configured escalation role remains distinct; rung 2 is owned by #397.

13.10.4 Gateway decisions contract

Authenticated `POST /v1/decisions` accepts `{model, state, questions, route_binding}`. `model` is an exact registry ID, `state` an object and `questions` a nonempty typed question object; supported question kinds are `choice`, `noul`

and `score`. Auto H uses its frozen `tier` choice question. Missing enforcement, absent/mismatched binding, ineligible/stopped model, unsupported disclosure or capacity overflow refuses before upstream dispatch (`src/optimus_gateway/decisions.py:32–83`).

`admit_decision` requires the approved effective hash, classifier eligibility and the route estimator. The complete canonical state/questions estimate must fit `min(8192, route windows)`, inclusive. Decisions reserve zero output tokens; the registry's positive classifier reserve remains a validation requirement. The route sends approved provider controls, including disabled fallback (`src/optimus_gateway/model_policy.py:291–341`).

The handler makes exactly one upstream attempt through `create_decision_once`, which posts to the aggregator's `/systemone` endpoint. There is no resend, including after rejection or uncertain delivery. Attempt outcomes and available usage are retained. The parser requires matching typed answers, valid choice/range values, IDs/model and nonnegative finite reported usage/cost; malformed choice types become unreadable uncertain replies (`src/optimus_gateway/decisions.py:85–117`; `src/optimus_gateway/upstream_client.py:303–342`, `:362–404`).

`check_decision_reply` accepts the requested model ID or its dated snapshot. An admitted provider-family route requires its recorded provider name; an empirically admitted route requires reported input no greater than the admitted estimate. System One supplies no finish status, router-attempt count or pipeline facts, so those completion-route checks cannot run here. The limitation is recorded in the classifier's admission data. A billed reply refusal preserves usage and stops that exception model; uncertain delivery also stops it. A proven rejected attempt does not acquire an invented charge (`src/optimus_gateway/model_policy.py:344–373`; `src/optimus_gateway/decisions.py:93–116`; `src/optimus_model_policy/auto_overlay.yaml:80–98`).

13.10.5 Classification settlement and cancellation

The classifier call runs in the turn's worker under its registered Gateway dispatch lease. A refused lease sends nothing. Every attributable physical attempt is offered as a `StageReceipt` with `stage="classification"` to that turn's existing settlement; receipt and integrity failures take precedence over the routing-failure rule. Known costs remain charged; unknown accounting stays unknown, including after teardown. Late receipts do not authorize an answer (`src/optimus/acp/auto_hybrid.py:205–285`; `src/optimus/usage/turn_settlement.py:38–39`).

The adapter refuses an accounting-integrity failure as `ACCOUNTING_INTEGRITY`. Cancellation or abandonment after classification prevents subsequent planning/answer dispatch and follows the existing turn settlement/finalization path. Classification is not part of the context meter's planning/answer input reading and adds no product monetary ceiling.

13.10.6 Notice and routing record

For a routed turn the host attempts one `Auto: <tier> -> <model>` message before planning/answer work, with reasons for confidence route-up, unavailable tiers, classifier failure or default fallback. It requests confirmed flushing, records whether delivery succeeded and contains notification failure. The notice is informational and is not stored in canonical history (`src/optimus/acp/auto_composition.py:160–171`; `src/optimus/acp/spec.py:1876–1888`).

The `auto_routing` event records arm H, disposition (`routed`, `refused`, `cancelled`, `abandoned`), source of the answer, predicted/classified/final tier, confidence band, route-ups, fallback, selected model/role, classifier-attempt outcome, reason and notice delivery. Unknown fields remain null. It contains no prompt, contract, answer text or raw confidence. The adapter guards emission at most once per admitted turn, with first write winning; telemetry failure is contained, so durable delivery is not guaranteed (`src/optimus/acp/auto_composition.py:174–203`; `src/optimus/acp/spec.py:1890–1932`; `src/optimus/telemetry/events.py:569–588`).

The receipt owns the classifier's actual cost; the routing record is a projection, not a second debit. Auto grants no tool permission or mutation authority. Exact artifact approval, mode enforcement, request capacity, disclosure and existing guards remain required.

13.10.7 Evidence and limitations

§14 separates source inspection, unit composition tests, reported suites/CI, the classifier pilot and real acpx slices. ADR-028 accepts H with its limitations: the measured 4.7% rates do not establish population rates below 5%; the previous-tier signal is

barely tested (#425); routing savings depend on models/workload. A different judge mechanism is separately owned by #426. Round 2 remains under #395; neither Auto's shipment nor these successor documents declare the whole plan complete.

14. Contract status: implemented, verified, active (correction of 2026-10-15)

This section classifies the executable contracts this document specifies with carried statements and baseline rows at the inspected source `91fd64ac78025efe1076bddb106ff89e6adb2030` (origin/main, 2026-10-15). Implementation, verification and activation are independent fields. Conceptual or retained-prototype interfaces are labelled as such where they appear; this table is the collected view. It changes no accepted contract. The correction map filed with the sources (`docs/sources/capability-evidence-correction/correction-map.md`) lists each change in that correction and its source reference.

Agent-mode EDIT and its execution-path corrections retain main `57e2069414fef32c6a6e4de42bd0dc7fb9f4f1ac` provenance (2026-10-24, including PR #413 and PR #414). Registry, production-construction and Auto statements explicitly updated here describe main `bf440cba5c184f8bca7ed8af0a0ae6bb824f14f1` (2026-10-27). Other contracts retain their historical pins and have not been generally reverified at `bf440cb`.

The 2026-10-18 Plan 12.2 ordinary shipping amendment (`docs/sources/plan-12-2-ordinary-shipping/architecture-amendment.md`) remains controlling for unrefreshed §13/§14 material, including compaction, profiles, maintenance and historical evidence. The refreshed registry/construction rows below supersede their inactive descriptions only. ADR-024 governs current interactive loop control; retained loop types and numerical bounds are historical. Source presence, reported verification and launch authorization remain separate.

Contract (section)	Implementation at the pin	Verification	Activation
Gateway routes `/v1/responses`, `/v1/chat/completions`, four typed tools, `/v1/observability/traces` (§0, §D)	Implemented (`optimus_gateway/server.py`, `tool_handlers.py`, `observability.py`)	Unit tests; `requires_gateway` and `requires_phoenix` live tiers	Active
Strict loopback settings (`OptimusGatewaySettings`, §9C)	Implemented (`optimus/config/gateway.py`, `_validate_loopback_gateway_url`; the `validate_loopback_gateway` listing on the carried §9C page is illustrative)	Unit tests	Active
Upstream transport `UrllibOpenAICompatibleClient` (§6, §6.1)	Implemented (`optimus_gateway/upstream_client.py`)	Unit tests	Active
Vercel optional endpoint (§0, §6)	Specified; not established by inspection	None	Inactive
Deterministic OpenRouter plugin search (§9C.1)	Specified target; Gateway search is `TavilyWebToolProvider`	Spike evidence only (2026-07-27, outside the Gateway)	Inactive
Package/OSV construction independent of search configuration (§9C.4)	Specified; not implemented (single bundle gated by `TAVILY_API_KEY`)	None	Not applicable
Gateway server-side policy revalidation (§9D)	Implemented (`optimus_gateway/tool_policy.py`, `tool_handlers.py`); the `authorize_tool_call` listing is illustrative	Unit tests	Active
Evidence ledger and USD run totals (§9E, §9E.1)	Implemented (`optimus/evidence`); rename complete, fields `ledger_run_total_cost_usd` and `ledger_run_total_billing_units`	Unit tests; `requires_redis` tier	Active

Contract (section)	Implementation at the pin	Verification	Activation
Usage accounting (§10)	Implemented (`UsageAccountingService.record_gateway_usage` in `optimus/usage/accounting.py`); the `record_usage` listing is illustrative	Unit tests	Active
RedisTimeSeries retention and idempotent creation (§10.1)	Implemented (`optimus/telemetry/redis_adapter.py`, 2,592,000,000 ms, `TS.CREATE` then `TS.ALTER`)	Unit tests; `requires_redis` tier	Active
Completion-request retry policy (§4A, §6, §13.4)	Existing host policy; enforced Gateway limit `MAX_ROUTE_ATTEMPTS = 2`, with the restricted resend conditions in §13.4	Historical tests retain their pins; source limit inspected at `bf440cb`	Enforcement active. Auto's separate decisions route permits one upstream attempt and no resend
ExecutionMode PLAN/CHAT/AGENT and mutation assertion (§4)	Implemented (`optimus/runtime/modes.py`; `MutationGuard` and `assert_mutation_allowed` in `optimus/runtime/mutation.py`, used by `optimus/gates/mutation_flow.py`)	Unit and integration tests	Active
`CodeGenerationScope`, `ExecutionStrategy`, `RigorLevel`, `AssumptionLedger`, `ToolResponse`, `ASTOptimizationInput`, `ADLParserStrategy` (§4, §5)	Retained design contracts; not present as named types under `src/` (`MULTI_FILE_CHANGESET` exists as a permission impact class)	None as named	Inactive
Composite fitness engine scaffolding (§7, carried page)	Illustrative scaffolding; the implemented runner is `optimus/gates/fitness.py`; the placeholder architecture and metric gates have no PyTestArch or maintainability-index implementation	Unit tests of the runner	Active for composition-supplied checks; not composed on the ACP WRITE/EDIT path at `57e2069`, which has no pre-write fitness check
Agent-mode EDIT directive (§4B)	Implemented at `57e2069` (`agent/directives.py`, `agent/edits.py`, `agent/models.py`, `agent/planning_loop.py`, `agent/runner.py`, `agent/tools.py`, `agent/prompts.py`, `agent/workspace_context.py`, `loops/tools.py`, `tools/line_endings.py`, `tools/mutation_tools.py`, `runtime/mutation.py`, `telemetry/events.py`, `acp/dispatcher.py`, `acp/spec.py`)	Unit and real-AgentRunner tests; Windows suites and PR CI at `92df22f`, `a970189` and `87b7b06`; post-merge CI at `57e2069`. One live acpx success at `92df22f` only; failure paths have unit/runner evidence only. See EDIT evidence below	Active in Agent mode at `57e2069`

Contract (section)	Implementation at the pin	Verification	Activation
Guardrail contracts: `PermissionPolicy`, `PreToolGuard`, `CommandSafetyValidator`, `ConfigTrustScanner`, `MCPTrustRegistry` (§12A, §12B)	Implemented (`optimus/guardrails`)	Unit tests (`tests/unit/guardrails`)	Active
Borderline classifier and loop completion evaluator model calls (§12A, §12C)	`GatewayCompletionEvaluator` implemented (`optimus/loops/completion.py`) behind `CompletionEvaluatorProtocol`; production composition uses `DeterministicCompletionEvaluator` and nothing in `src/` constructs the Gateway evaluator; classifier reserved	Unit tests (`tests/unit/loops/test_completion.py`); none live	Inactive (not composed in production)
Bounded loops: `GoalLoopController`, `LoopBudgetPolicy`, `LoopStopReason` (§12C)	Implemented (`optimus/loops`, `optimus/agent/planning_loop.py`); `max_budget_usd` defaults `None`	Unit and integration tests	ACP planning bounds active (3 turns, 30 minutes, 2 repeated failures)
Curated skills: `SkillRegistry`, `SkillManifest`, `SkillTrustPolicy` (§12D)	Implemented (`optimus/skills`)	Unit tests	Active
`make_history_snapshot`, `HistoryRevision`, `ContextEngine.prepare_view`, `StrategyParameters`, `ViewLimits` (§13.1)	Implemented; production attachment is composed at `bf440cb`	Historical cf9bb9d evidence retains its pin; other Plan 12.2 evidence remains under the amendment	Available through ordinary/Auto attachment when an eligible summarizer exists
Whole-turn maintenance and checkpoints (§13.2)	Existing engine and host maintenance reused by Auto; no new compaction strategy	Historical cf9bb9d evidence; Auto unit composition test exercises compaction	Production construction exists; unrefreshed policy/details remain under the amendment
`ContextAttachment`, `AttachedTurn`, `probe_floor`, `max_repacks` (§13.3)	Production constructor in `optimus/acp/context_attachment.py`; Auto supplies selected-model limits	Code inspection at `bf440cb`; unit composition evidence, separately qualified below	Ordinary/Auto construction; no test-profile authority is inherited
Trusted `RegistrySnapshot`, eligibility and final capacity guard (§13.4)	Enforcement active at `bf440cb`; per-entry admissions and trusted operator override (ADR-025); ADR-026 guardrail predicate	Code inspection; registry/override tests; reported PR #428 evidence below	Active under approved effective hash; classifier uses its separate 8192-token request bound
`StageReceipt` exactly-once settlement and D6 (§13.5, §13.6)	Implemented; product dollar stops removed, merged via PR #276	Offline-verified at cf9bb9d	Active
ACP projection, meter and capacity messages (§13.7, §13.9)	Implemented (`optimus/acp/spec.py`, `gateway/attempts.py`)	Offline-verified at cf9bb9d	Capacity text selectable once enforcement is active
`--plan12-test-profile`, `TEST_PROFILE_GATEWAY_NOT_STARTED`, `GatewayModelPolicy` (§13.9)	Implemented (`optimus/acp/__main__.py`, `optimus_model_policy/test_profiles.py`, `optimus_gateway/model_policy.py`); profile A admits no model, profile B is declared unavailable	Offline-verified at cf9bb9d with a synthetic eligible profile	Test-composition only

Contract (section)	Implementation at the pin	Verification	Activation
Auto composition and launch binding (§13.10.1)	`auto_snapshot`, `compose_auto_composition`, `_model_registry` and `_composition`; independent Gateway composition	Launch/override unit tests; reported PR #428 CI	Opt-in Auto only; separate approved composition required
`AutoRouter`, frozen input and one host stop set per launch (§13.10.2–§13.10.3)	Implemented in `agent/auto_contract.py`, `acp/auto_composition.py`, adapter and runner	Routing/stop tests; real configured-server unit composition test	Active in approved Auto launches; planning/answer model fixed per turn
H judge (§13.10.2)	`acp/auto_hybrid.py`; rules then configured Jev; confidence validation and failure rule	Pilot accepted by ADR-028; unit tests; reported live complex slice at `df7dc3c`	Auto's shipped judge; no struggle-escalation implementation
`POST /v1/decisions`, `admit_decision`, `check_decision_reply` (§13.10.4)	Authenticated typed route; one upstream attempt; distinct System One limitations	Gateway decision/parser tests; reported live complex slice	Requires enforced classifier-eligible registry and exact binding
`classification` receipts and turn settlement (§13.10.5)	Registered dispatch lease; receipt-first accounting and integrity refusal	Classifier/accounting/cancellation tests; composition test	Active for Auto classifier attempts; no fabricated zero or duplicate debit
Auto notice and `auto_routing` event (§13.10.6)	Informational flushed-notice attempt; content-free first-write routing record	Notice/cancellation/concurrency tests; three reported live slices	Auto turns; notification/telemetry failures contained, no durable-delivery guarantee
`optimus-trust run-gateway --auto` (§13.10.1)	CLI flag; durable workspace approval must bind registry and composition	Launch CLI unit tests	Supported for a manually managed Auto Gateway
Package extraction and publication of `context_engine` as a wheel (§13.1)	Buildable; independent wheel proof at cc6959b carried to cf9bb9d over unchanged inputs	Historical proof only; not rebuilt at the pin	Not published

Auto evidence. The refreshed implementation statements use main `bf440cba5c184f8bca7ed8af0a0ae6bb824f14f1`. Unit sources cover composition/approval, override, routing, model stops, H, decisions, notices, accounting and cancellation. `tests/unit/acp/test_auto_launch_end_to_end.py:134–186` drives the real configured server over NDJSON through the adapter and AgentRunner, replacing external dependencies. Its stage-set assertions establish stage presence, not exact receipt counts or total costs. It is unit composition evidence, not an independently authored live ACP client.

PR #428 (https://github.com/vibhanshu-agarwal/optimus-cost-agent/pull/428) reports one Windows full suite with coverage at `3420eb2`: 10,107 passed, 50 skipped, 101 deselected, 1 warning; required product gates passed. This is not a Windows full-suite result for the later Linux test-isolation correction or merge of main. Preserve the PR's first Linux run with three failures, its correction at `236a3c4`, and the two repository-layout failures observed with unmerged index entries. At final PR head `df7dc3c52dbc64ec5d18d27e05766c1953715853`, guardrails run 40469959706 (https://github.com/vibhanshu-agarwal/optimus-cost-agent/actions/runs/40469959706) completed successfully. No workflow runs were returned for merge `bf440cb` during this authoring pass; no post-merge CI pass is inferred.

#395 comment 6437429914 (https://github.com/vibhanshu-agarwal/optimus-cost-agent/issues/395#issuecomment-6437429914) reports real acpx 0.12.0 Agent-mode live slices at `df7dc3c`: easy through rules to Luna, medium through rules to Gemini, complex through Jev `/v1/decisions` to Gemini; 3/3 passed, reported total USD 0.005858. The comment records notices, routing records, completed accounting, file outcomes and cleanup. Haiku fallback was not exercised. It also records Jev's dated snapshot displaying as `unrecognized` in the Gateway log while the decision reply check accepted it. These are reported PR-head results; the external raw bundle was not independently inspected for this authoring pass and is archival provenance, not a project dependency.

The filed pilot bundle and ADR-028 retain the selected judge's evidence and limitations. They do not prove population error rates below 5%, universal task suitability or savings. Round 2 and the document successor retain custody under #395; this section does not close the whole plan.

EDIT evidence. Unit and runner tests (`tests/unit/agent/test_edit_directives.py`, `test_edits.py`, `test_runner_edit.py`, through the real `AgentRunner`). PR #411 (https://github.com/vibhanshu-agarwal/optimus-cost-agent/pull/411): Windows full suite failed at `271c16a` with 4 probe import-closure failures; final run at `92df22f`: 9,430 passed, 49 skipped, 101 deselected, 1 warning. guardrails run 40069592519 (https://github.com/vibhanshu-agarwal/optimus-cost-agent/actions/runs/40069592519): pull_request, head `92df22f`, success. #407 comment 6382141300 (https://github.com/vibhanshu-agarwal/optimus-cost-agent/issues/407#issuecomment-6382141300): live acpx 0.12.0, Haiku 5.5, 2026-10-24, exact-byte edit of a 110,167-byte file, USD 0.0014817. Failure paths have unit/runner evidence only, not live evidence. These are PR-head results, not merge-commit reruns.

PR #413 (https://github.com/vibhanshu-agarwal/optimus-cost-agent/pull/413), head `a970189`, merged as `159a712`: its reported Windows full suite with coverage gave 9,610 passed, 49 skipped, 101 deselected, 1 warning. guardrails run 40093667921 (https://github.com/vibhanshu-agarwal/optimus-cost-agent/actions/runs/40093667921): pull_request, head `a970189`, success. Codex CLI round 55 accepted that head for its full-suite gate; rounds 48–55, including the earlier NOT ACCEPTED reviews, remain on #410 (https://github.com/vibhanshu-agarwal/optimus-cost-agent/issues/410#issuecomment-6384154573).

PR #414 (https://github.com/vibhanshu-agarwal/optimus-cost-agent/pull/414), head `87b7b06`, merged as `57e2069`: its reported Windows full suite with coverage gave 9,431 passed, 49 skipped, 101 deselected, 1 warning; its Codex CLI checkpoint review reported APPROVE. guardrails run 40093673312 (https://github.com/vibhanshu-agarwal/optimus-cost-agent/actions/runs/40093673312): pull_request, head `87b7b06`, success. The first Masterplan impact run 40093673253 failed for a missing declaration (https://github.com/vibhanshu-agarwal/optimus-cost-agent/actions/runs/40093673253); replacement run 40094074645 succeeded at the same head (https://github.com/vibhanshu-agarwal/optimus-cost-agent/actions/runs/40094074645).

Post-merge main `57e2069`: guardrails run 40096867014 (https://github.com/vibhanshu-agarwal/optimus-cost-agent/actions/runs/40096867014) and Work status page run 40096866988 (https://github.com/vibhanshu-agarwal/optimus-cost-agent/actions/runs/40096866988), both push events at that merge head, succeeded. These CI results are separate from the Windows PR-head suites. Neither PR #413 nor PR #414 adds live evidence; the only live run above remains at `92df22f`.

Activation and evidence boundaries. The predecessor's production-absence checklist is historical; production construction and registry enforcement exist at `bf440cb`. The Plan 12.2 amendment remains controlling for unrefreshed qualification and evidence statements. Empirical admission remains explicitly unverified and retains its first-call risk. Auto requires its own approved registry/composition and launch prerequisites (§13.10); neither old test-profile acceptance nor this document authorizes a launch, paid call or service change. Reviewer behavior, savings reporting and automatic struggle escalation retain their separate owners.